Sparse4D: Sparse-Based End-to-End Multi-Sensor Temporal Perception

Xuewu Lin, Zixiang Pei, Keyu Li, Tianwei Lin, Lichao Huang, Chenyao Yu, John See, Zhizhong Su, and Cong Yang

***Abstract*—In the field of autonomous driving, the structural design of perception models is of paramount importance. Unlike mainstream BEV-based algorithms, we propose a novel end-to-end sparse perception framework, Sparse4D, to achieve better performance and higher efficiency. Starting from the sparsity of perception results, we define an instance that decouples implicit features and explicit anchors, using the instance as the core for feature fusion to accomplish perception tasks. For spatial modelling, we develop a novel operator called deformable aggregation, which enables the transfer of information from the dense image feature to the sparse instance. For temporal modelling, we design a recurrent instance feature propagation structure, which not only realizes long-term feature fusion but also ensures the computational efficiency of the temporal module. Lastly, we explored the performance of Sparse4D in multi-object tracking tasks and proposed a minimalist joint detection and tracking model. We conduct extensive experimental validation of Sparse4D on the nuScenes benchmark. Sparse4D achieved state-of-the-art performance in multi-camera 3D detection and tracking tasks, and it also outperformed other algorithms in terms of training and inference efficiency. Furthermore, we extended Sparse4D to a multi-modal model, achieving excellent performance and better generalization.**

***Index Terms*—Autonomous driving, end-to-end perception, sparse, 3D detection, multi-object tracking.**

I. Introduction

In THE field of autonomous driving, end-to-end perception holds significant research value for better realizing data-driven approaches [1]. Notably, an end-to-end perception model should achieve the following: 1) feature fusion from multiple sensors (including multiple cameras and modalities), 2) temporal feature fusion, and 3) produce directly usable perception results, which includes detection positions, categories, and motion trajectories [2], [3], [4], [5]. Moreover, adopting an end-to-end approach can significantly reduce the dependency of perception models on post-processing modules. This shift allows for effective handling of long-tail cases through increased data availability, thereby enhancing the overall performance of the system [6].

Existing end-to-end perception algorithms primarily adopt a BEV (Bird's Eye View) perception architecture. This involves projecting multi-sensor features onto a unified BEV vector coordinate system, where multi-sensor feature fusion and temporal fusion are performed [7]. These algorithms require the construction of dense BEV features, which can limit the model's computational efficiency and complicate deployment [8]. Additionally, autonomous driving scenarios necessitate an extensive perception range, often exceeding 200 meters, which increases the computational load of BEV-based algorithms. The linear increase proportional to the expansion of the perception range makes it challenging to balance accuracy with efficiency [9]. In contrast, sparse perception models such as DETR3D [10] and its extensions do not require the construction of a dense vector space, resulting in better computational efficiency and reduced sensitivity to the perception range. However, these models currently underperform compared to their BEV-based counterparts, demonstrating a significant performance gap. Finally, it is worth noting that existing end-to-end perception models struggle with 3D multi-object tracking, hindering their ability to achieve high-quality joint detection and tracking, which is essential for mass production.

Given these challenges, we propose a novel perception model called Sparse4D for end-to-end multi-sensor temporal perception. Inspired by existing sparse algorithms [10], [19], [20], Sparse4D eliminates the constraints of the BEV vector space and utilizes a sparse instance representation to achieve multi-source feature fusion. In particular, we design a multi-view fusion operator known as deformable aggregation. This operator merges multi-view features through 3D key points, projection transformations, and weighted sampling. Additionally, we also optimize deformable aggregation with a rational implementation, significantly enhancing both the training and inference speed of Sparse4D and reducing memory consumption. For temporal fusion, we introduce a recurrent method for instance temporal feature fusion. This approach not only reduces the computational complexity of temporal fusion from $O(T)$ to $O(1)$, resulting in significant improvements in inference speed and memory usage, but also enables the fusion of long-term information, leading to more pronounced performance improvements. As shown in Fig. 1, our comprehensive experiments illustrate that Sparse4D greatly enhances the 3D detection performance of sparse approaches and surpasses all existing BEV-based algorithms.

Received 27 March 2025; revised 20 February 2026; accepted 25 April 2026. Date of publication 28 April 2026; date of current version 6 August 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62473276 and Grant 62573309, in part by the Natural Science Foundation of Jiangsu Province under Grant BK20241918, and in part by the Research Fund of Horizon Robotics under Grant H230666. Recommended for acceptance by J. Huang. *(Corresponding author: Cong Yang.)*

Xuewu Lin is with the BeeLab, School of Future Science and Engineering, Soochow University, Suzhou 215006, China, and also with Horizon Robotics, Beijing 100094, China.

Zixiang Pei, Keyu Li, Tianwei Lin, Lichao Huang, and Zhizhong Su are with Horizon Robotics, Beijing 100094, China.

Chenyao Yu and Cong Yang are with the BeeLab, School of Future Science and Engineering, Soochow University, Suzhou 215006, China (e-mail: cong.yang@suda.edu.cn).

John See is with the School of Mathematical and Computer Sciences, Heriot-Watt University Malaysia, Putrajaya 62200, Malaysia.

Digital Object Identifier 10.1109/TPAMI.2026.3688545

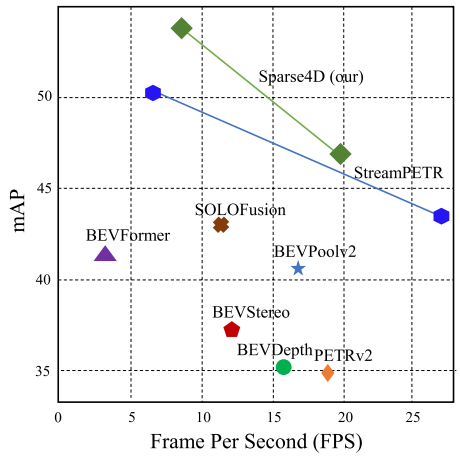


Fig. 1. Inference efficiency (FPS) - perception performance (mAP) on nuScenes [11] validation dataset of state-of-the-art algorithms (StreamPETR [12], SOLOFusion [13], BEVFormer [14], BEVPoolv2 [15], BEVStereo [16], BEVDepth [17], PETRv2 [18]). Our proposed Sparse4D (marked in green) offers advantages in both detection and inference.

To advance the end-to-end capabilities of the perception system, we explore the integration of 3D multi-object tracking tasks into the Sparse4D framework. This integration allows for the direct output of object motion trajectories. Unlike tracking-by-detection methods [21], we eliminate the need for data association and filtering by consolidating all tracking functionalities within the detector. Moreover, in contrast to existing joint detection and tracking methods [22], [23], [24], our tracker does not require modifications to the training process or loss functions. It also does not necessitate the provision of ground truth IDs but still achieves a predefined instance-to-tracking regression. Our implementation maximally combines the roles of the detector and the tracker, requiring no changes to the detector's training process or any additional fine-tuning. Finally, Sparse4D is flexible and can be easily extended to work with LiDAR and multi-modal models. Without bells and whistles, Sparse4D can deliver state-of-the-art performance in LiDAR point cloud and multi-modal detection.

Succinctly, the main contributions of this work are as follows: 1) We propose a novel perception framework called Sparse4D, which facilitates both multi-sensor spatial fusion and temporal fusion, enabling seamless end-to-end environmental perception. 2) We implement end-to-end multi-object tracking based on Sparse4D, featuring a highly simplified tracker architecture. 3) We conduct extensive experimental validation using the publicly available nuScenes [11] dataset. Our results demonstrate that Sparse4D excels in 3D detection and multi-object tracking while offering improvements in both training and inference efficiency.

II. Related Works

We now present a concise survey of existing detection, tracking, and fusion approaches in multi-view, multi-object, and multi-modal scenarios. For a more thorough treatment of end-to-end and multi-sensor perception, recent compilations by Chen et al. [1] offer sufficiently good reviews.

A. Multi-View 3D Detection

Dense algorithms are the primary research direction of multi-view 3D detection, which utilises dense feature vectors for view transformation, feature fusion or box prediction. Currently, BEV-based methods are the main focus of dense algorithms. For instance, BEVFormer [14] adopts deformable attention to complete the BEV feature generation and dense spatial-temporal feature fusion. BEVDet [25] uses lift-splat operation [26] to achieve the view transformation. On the basis of BEVDet, BEVDepth [17] adds explicit depth supervision, which significantly improves the accuracy of the detection. BEVStereo [16] and SOLOFusion [13] introduce temporal stereo technology into 3D detection, further improving the depth estimation effect. PETR [18], [27] utilises 3D position encoding and global cross attention for feature fusion, while global cross attention is computationally expensive.

Despite the advantages of BEV methods, there are notable limitations in their practical deployment: 1) The transformation of images to the BEV perspective requires dense feature sampling or rearrangement, which can be complex and computationally expensive for deployment on low-cost edge devices. 2) The maximum perception range is constrained by the size of the BEV feature map, making it challenging to balance perception range, efficiency, and precision. 3) The height dimension is compressed in BEV features, leading to a loss of texture cues. As a result, BEV features may be inadequate for certain perception tasks, such as sign detection. In contrast to BEV-based methods, sparse-based algorithms, such as DETR3D [10], do not require a dense perspective transformation module. Instead, they directly sample sparse features for 3D anchor refinement, addressing the limitations mentioned above. DETR3D, for instance, performs feature sampling and fusion based on sparse reference points. Graph DETR3D [19] builds on this by introducing a graph network to enhance spatial feature fusion, particularly in multi-view overlapping regions. However, their effectiveness is somewhat limited since DETR3D samples features from only a single 3D reference. SRCN3D [20] improves upon this by using RoI-Align [28] to sample multi-view features, but it lacks efficiency and struggles to align feature points accurately from different views. Furthermore, existing sparse 3D detection methods have not leveraged the benefits of rich temporal context, resulting in a significant performance gap when compared to state-of-the-art BEV-based methods [7].

Differently, our proposed Sparse4D is a sparse multi-view 3D detection algorithm with temporal context fusion, which can efficiently and effectively align spatial and temporal visual cues to achieve precise 3D detection. The rationale behind this is that, by decoupling image features and structured anchor features, Sparse4D enables a highly efficient transformation of temporal features, thereby facilitating temporal fusion solely through the frame-by-frame transmission of sparse features.

B. End-to-End Multi-Object Tracking

Most multi-object tracking (MOT) methods use the tracking-by-detection framework [21]. They rely on detector outputs to perform post-processing tasks like data association and trajectory filtering, leading to a complex pipeline with numerous hyper-parameters that need tuning. These approaches do not fully leverage the capabilities of neural networks. To integrate the tracking functionality directly into the detector, GCNet [22], TransTrack [23] and TrackFormer [24] utilize the DETR [29] framework. They implement inter-frame transfer of detected targets based on track queries, significantly reducing post-processing reliance. MOTR [30] advances tracking to a fully end-to-end process. Extended works [31] address the limitations

in the detection query training of MOTR, resulting in a substantial improvement in tracking performance. MUTR3D [32] applies this query-based tracking framework to 3D multi-object tracking. These end-to-end tracking methods share some common characteristics: 1) During training, they constrain matching based on tracking objectives, ensuring consistent ID matching for tracking queries and matching only new targets for detection queries. 2) They use a high threshold to transmit temporal features, passing only high-confidence queries to the next frame.

Our proposed Sparse4D diverges from existing methods: it does not need to modify detector training and inference strategies since the MOT functionalities are encoded adequately along with the detector. Moreover, it does not require ground truth for tracking IDs, as they are simplified in our detection framework.

C. Multi-Modal Fusion

LiDAR and vision-based models each have distinct strengths and weaknesses in the realm of autonomous driving [33], [34], [35], [36], [37], [38]. Thus, fusing these two modalities is a major focus of current research [39], [40], [41], [42], [43]. In recent years, multi-modal fusion approaches have gradually shifted from dense-based to query-based strategies in order to reduce computational overhead and capture accurate object features. Notably, FUTR3D [44] achieves multi-modal fusion by initializing a set of 3D reference points to sample corresponding information from different modalities. TransFusion [45] utilizes a two-stage pipeline, first generating a set of queries using LiDAR features and then refining them with image features. CMT [46] encodes positional information from multiple modalities into the queries by employing position encoding. SparseFusion [47] takes this further by leveraging sparse representations for multi-modal fusion.

Inspired by these approaches, our proposed Sparse4D is designed to be flexible and easily adaptable to both LiDAR and multi-modal domains. Our qualitative and quantitative experiments demonstrate that Sparse4D achieves impressive results, including a remarkable mean Average Precision (mAP), uScenes Detection Score (NDS), and Average Multi-Object Tracking Accuracy (AMOTA) on the nuScenes test dataset [11], surpassing certain metrics compared to detection models that utilize LiDAR, such as TransFusion [45].

D. Sparse4D Series

Sparse4D series involves Sparse4D v1 [48], v2 [49], v3 [50]. This official version is the first formal publication of the Sparse4D series' research achievements. It does not simply "assemble" previous modules. Instead, it re-designs the entire framework to resolve compatibility and efficiency bottlenecks when integrating these modules. For example, we optimize the interface between Deformable Aggregation (from Sparse4D v1) and Recurrent Temporal Fusion (from Sparse4D v2) by introducing a unified feature encoding layer, which reduces inter-module feature conversion overhead by 35% compared to the separate use of the two modules. Moreover, it streamlines the implementation of joint detection and tracking, allowing for stable tracking without any modifications to the detector or tracking labels.

It should be noted that deformable aggregation and recurrent instance feature propagation have appeared in prior multi-view 3D detection works (e.g., Far3D [51], FB-BEV [52]). Sparse4D introduces a unified sparse 4D representation that fundamentally differs in both formulation and usage. Unlike prior methods that rely on dense BEV grids or frame-wise instance propagation, Sparse4D maintains a persistent set of sparse 4D instances that jointly encode spatial and temporal information across frames.

TABLE I
CRUCIAL SYMBOLS AND THEIR DESCRIPTIONS INVOLVED IN THIS PAPER

Symbol	Description	Sections
I	Feature map	3
N	Number of sensors	3
S	Number of scales	3
A	Anchor of a bounding box	3.1
(x, y, z)	3D location of a bounding box	3.1
(w, l, h)	Size of a 3D bounding box	3.1
yaw	Rotation of a bounding box	3.1
(v_x, v_y, v_z)	Velocity in each direction	3.1
E	Position embeddings	3.1
$\Psi(\cdot)$	Multilayer perception	3.1
P	A series of 3D keypoints	3.2.1
f	Feature vectors	3.2.1
Bilinear$(\cdot)$	Bilinear interpolation	3.2.1
W	Weight	3.2.1
Cam	Camera parameters	3.2.1
C	Number of channels	3.2.1
K	Number of keypoints	3.2.1
G	Number of groups	3.2.1
$\Phi(\cdot)$	Camera parameter encoding	3.2.1
F	Fused instance feature	3.2.1
Project$(\cdot)$	Anchor projection	3.3
t	Time stamp for steps and frames	3.3
$\mathbf{T}$	Transformation matrix of the ego vehicle	3.3
$\mathbf{R}$	Rotation matrix of the ego vehicle	3.3
D	Sensor data	3.3
ΔA	Random noise	3.6.1
M	Group number of noising instances	3.6.1
M'	A temporary group number	3.6.1
L	Overall loss of Sparse4D	3.6.4
L_{cls}	Classification loss	3.6.4
L_{box}	Bounding box regression loss	3.6.4
L_{depth}	Depth estimation loss	3.6.4
L_{qua}	Quality loss	3.6.4
λ	Weight terms	3.6.4
i, j, k, n, s	Temporary and index numbers	3

III. SPARSE4D

Fig. 2 presents the overall structure of our proposed Sparse4D, which conforms to an encoder-decoder structure. For better understanding, some essential symbols and their descriptions are summarized in Table I. The feature encoder is used to extract image and LiDAR point cloud features. For different modalities, we use encoders with independent parameters. In terms of sensors within the same modality, we use encoders with shared parameters. For instance, we use only one backbone and neck for multi-view images. Given N sensor data at time t, the feature encoder extracts multi-sensor and multi-scale feature maps I_t:

$$I_t = \{I_{n,s} | 1 \le s \le S_n, 1 \le n \le N\}. \tag{1}$$

where S_n indicate the number of scales of the n_{th} sensor features. These feature maps are then fed into the decoder, which consists of one ($\times 1$) single-frame layer and five ($\times 5$) multi-frame layers, which are marked in grey in Fig. 2.

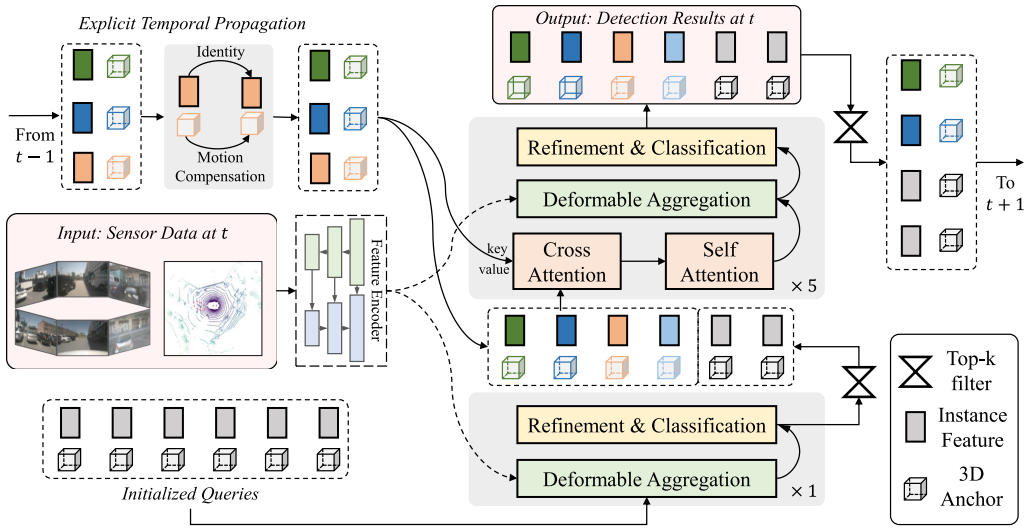


Fig. 2. Sparse4D framework. The inputs mainly consist of three components: sensor data at frame t, newly initialized instances, and propagated instances from the previous frame. The output is the refined instances (3D anchor boxes and corresponding features), which serve as the perception results for the current frame. Additionally, a subset of these refined instances is selected and propagated to the next frame. Best viewed in colour.

The decoder takes inputs not only from the sensor features but also from predefined instances (see Section III-A). The single-frame layer includes three sub-modules: Deformable Aggregation, Feedforward Network (FFN), and the output layer for Refinement and Classification. The custom-designed deformable aggregation (see Section III-B) aims to enable interaction between instances and sensor features. The multi-frame layers, in addition to the aforementioned sub-modules, also incorporate two multi-head attention layers [53]. The cross-attention module is used to enhance the fusion of temporal features, while the self-attention module facilitates direct feature interaction among instances. This way, the multi-level cascaded refinement structure in the decoder adequately utilizes sensor features to enhance instances iteratively (see Section III-C) and ultimately produce perceptual outcomes.

A. Formulation of Query Instances

Unlike the queries in DETR3D [10], we explicitly decompose each instance into two distinct components: anchor A and instance feature F. Anchor defines the structured priors associated with the target object, while instance feature encompasses the high-dimensional perceptual characteristics of the target object. For the 3D detection task, a 3D bounding box is defined as anchor A:

$$A = \{x, y, z, w, l, h, \sin yaw, \cos yaw, v_x, v_y, v_z\}. \quad (2)$$

where $(x, y, z), (w, l, h), yaw, v$ denote the location, size, rotation, and velocity of an object, respectively. All anchors are set in a unified 3D coordinate system (e.g. central LiDAR coordinate).

By employing this explicit anchor definition in (2), a wider spectrum of induced biases can be seamlessly incorporated into the network design, yielding three benefits. The first one is that anchors can be explicitly encoded as high-dimensional position embeddings E using an MLP (Multilayer Perceptron) Ψ:

$$E = \Psi(A). \quad (3)$$

which is proved to be more efficient than random initialization based on our preliminary experiments [48], [49], [50]. Secondly, by leveraging anchors, we can achieve more effective 3D key points, easing difficulties in network convergence. Thirdly, within the instant-level framework of temporal fusion, we can explicitly employ anchors and ego poses for temporal instance propagation.

B. Spatial Modelling

The quality of instance features has a critical impact on the overall sparse perception system. To address this, we introduce a deformable aggregation module to obtain high-quality instance features with sparse feature sampling and fusion (multi-view, multi-scale, multi-timestamp and multi-keypoint). We also introduce an efficient parallel implementation strategy to improve the training (also inference) efficiency and reduce the memory consumption of the deformable aggregation operation.

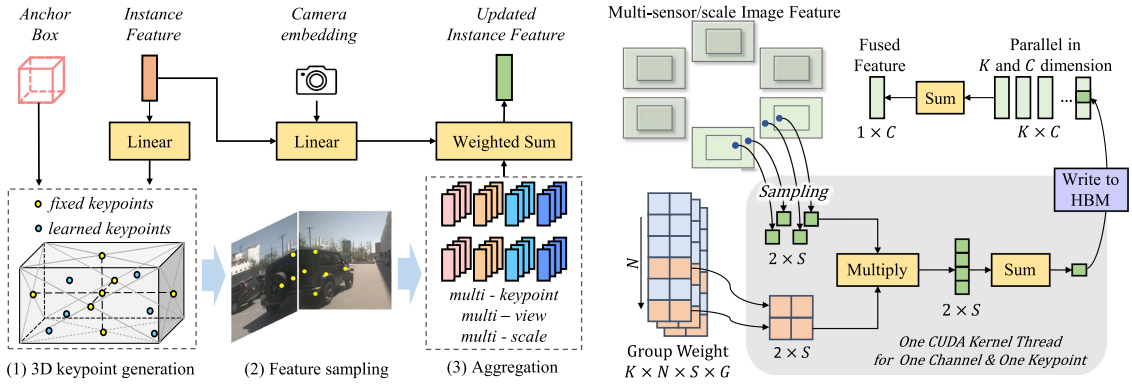
1) Deformable Aggregation: Deformable aggregation is designed to enhance the transfer of information from multi-view and multi-scale image features to instance-level details, resulting in more distinctive instance features. Unlike deformable attention [54], deformable aggregation begins with 3D key points, enabling the aggregation of features from multiple scales and perspectives within the feature map.

As shown in Fig. 3(a), for each instance, a series of 3D key points $P \in \mathbb{R}^{K\times 3}$ (with learnable and fixed) is generated, where K is the number of keypoints. These points are projected onto the feature map I (see (1)) using camera intrinsic and extrinsic parameters. Then, feature sampling is conducted with a bilinear interpolation, yielding multi-keypoint, multi-view and multi-scale feature vectors $f \in \mathbb{R}^{K\times N\times S\times C}$:

$$f_{k,n,s} = \textbf{Bilinear}\left(I_{n,s}, P_k^{\text{img}}\right). \quad (4)$$

where the P_k^{img} denotes the k_{th} projected keypoint of the instance, $k \in K$ (keypoints), $n \in N$ (views), $s \in S$ (scales). C denotes the number of channels. Instead of directly adding these feature up as instance feature, we aggregate these feature with dynamic weighting, while the weight W is predicted by:

$$W = \Psi\left(F + E + \Phi(Cam)\right) \in \mathbb{R}^{K\times N\times S\times G}. \quad (5)$$



(a) Pipeline of Deformable Aggregation Module (b) Parallel Implementation

Fig. 3. Illustration of our proposed Deformable Aggregation module. (a) Basic pipeline: We first generate multiple 3D keypoints inside the 3D anchor, then sample multi-scale/view image features for each keypoint, and fuse these features with predicted weight. (b) Parallel implementation: To further improve speed (training and inference) and reduce memory cost, we propose an efficient parallel implementation strategy, where feature sampling and multi-view/scale weighted sum are conjoined as a CUDA operation. HBM: High-Bandwidth Memory (see Section III-B2).

where Ψ and Φ are both MLP-based sub-modules, and G is the number of groups to divide features by channels. Here, Cam represents camera parameters. To enhance the camera generalization capability of Sparse4D, we introduce a camera parameter encoding network Φ. For instance, we can employ a 3D-to-2D projection matrix as the camera parameters for undistorted images.

With (5), we can aggregate channels of different groups with different weights. Similar to group convolution [55], we sum the weighted feature vectors along the keypoint, scale and view dimensions to generate fused instant feature F:

$$F = \sum_{k,n,s} W_{k,n,s} \times f_{k,n,s}. \tag{6}$$

2) Parallel Implementation: The fundamental computations of deformable aggregation consist of two essential components: bilinear interpolation sampling and weighted summation. Nevertheless, employing independent operators for these components can lead to repeated accesses to the GPU's high-bandwidth memory (HBM) and the accumulation of intermediate variables, particularly within the feature vector f in (4). As a result, it not only impacts both training and inference efficiency but also places a significant burden on memory resources. To address this problem, we encapsulate the feature sampling and scale/view dimension weighting in the basic Deformable Aggregation as a CUDA operation, directly outputting multi-point features in a single step (Fig. 3(b)). We refer to this optimization as Efficient Deformable Aggregation (EDA).

EDA demonstrates exceptional parallel performance, facilitating full parallelization in the K (keypoint) and C (channel) dimensions. The computational complexity for a single thread is merely $N \times S$. Additionally, in multi-view scenarios, our prior understanding dictates that a 3D point can be projected onto a maximum of two views. As a result, we can further curtail the computational complexity of a single-thread computation to $2 \times S$. This design leverages the parallel computing capabilities of GPUs and AI chips [56], significantly improving efficiency and reducing both memory consumption and inference time. EDA can serve as a versatile operation, suitable for various applications requiring multi-image and multi-scale fusion.

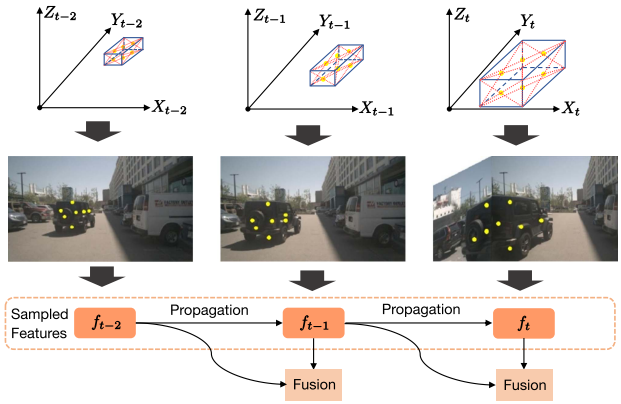


Fig. 4. General idea of our proposed recurrent temporal fusion.

C. Temporal Modelling

Temporal feature fusion can significantly enhance 3D perception capabilities [57]. In Sparse4D, we introduce a recurrent strategy to achieve efficient and extended feature fusion. As presented in Fig. 4, it is performed through instance-based temporal propagation, which entails transforming past-frame instances into the current frame. Benefit from our disentangled query instance design, we only require updating (anchor projection, followed by anchor encoding) the anchor box A of the query instance while maintaining the instance feature F unchanged (aka: $F_t = F_{t-1}$):

$$A_t = \mathbf{Project}(A_{t-1} + \triangle t \cdot v_{t-1}, \mathbf{T}_{(t-1)\to t}). \tag{7}$$

where $\mathbf{T}_{(t-1)\to t}$ represents the transformation matrix of the ego vehicle from last $(t-1)$ to current (t) frames. Thus, the cross-frame motion compensation consists of both object and ego motions.

By redefining the anchor in (2) and projection function in (7), Sparse4D can be adapted to tackle various perception tasks. For instance, in the context of a 3D detection task, the anchor is defined as a 3D bounding box and the projection function is:

$$A_{t-1} = \{x, y, z, w, l, h, \sin yaw, \cos yaw, v_x, v_y, v_z\}_{t-1}$$

$$[x, y, z]_t = \mathbf{R}_{(t-1)\to t}([x, y, z] + d_t[v_x, v_y, v_z])_{t-1} + \mathbf{T}_{(t-1)\to t}$$

$$
\begin{aligned}
&[w, l, h]_t = [w, l, h]_{t-1} \\
&[\cos yaw, \sin yaw, 0]_t = \mathbf{R}_{(t-1)\to t}[\cos yaw, \sin yaw, 0]_{t-1} \\
&[v_x, v_y, v_z]_t = \mathbf{R}_{(t-1)\to t}[v_x, v_y, v_z]_{t-1}. \qquad (8)
\end{aligned}
$$

where d_t is the time interval between frames t and $t-1$. $\mathbf{R}_{(t-1)\to t}$ represents the rotation matrix of the ego vehicle from time step $t-1$ to t. If 3D lane line detection is desired, the anchors can be defined as polylines, and the projection function would be the projection of each 3D point on the polyline.

In practice, the decoder of Sparse4D consists of two main components: a non-temporal layer and multiple temporal layers. The non-temporal layer takes newly initialized query instances as input and generates coarse detection outcomes for the current frame. From these outcomes, we select a subset of instances with the highest confidence scores. This subset, along with temporally propagated instances, serves as the input for the temporal layers. The temporal layers focus on achieving spatial-temporal feature fusion using cross-attention, self-attention, and our proposed EDA module. The final temporal layer produces refined instances, which represent the perception results for the current frame. These refined instances are then propagated to the next frame. This design approach eliminates the need to increase the instance count within the temporal decoder, effectively managing the integration of newly emerging objects.

D. End-to-End Tracking

As introduced in Section III-C, temporal modelling adopts a recurrent form, projecting instances from the previous frame onto the current frame as input. The temporal instances are similar to the tracking queries in query-based trackers, with the distinction that the tracking queries are constrained by a higher threshold, representing highly confident detection results. In contrast, our temporal instances are numerous, and most of them may not accurately represent detected objects in previous frames.

To extend from detection to multi-object tracking within the detection framework, we directly redefine the instance from a detection bounding box to a trajectory. A trajectory includes an ID and bounding boxes for each frame. Due to the setting of a large number of redundant instances, many instances may not be associated with a precise target and may not be assigned a definite ID. Nevertheless, they can still be propagated to the next frame. Once the detection confidence of an instance surpasses the threshold T, it is considered to be locked onto a target and assigned an ID, which remains unchanged throughout temporal propagation. Thus, achieving multi-object tracking is as simple as applying an ID assignment process to the output perception results. The lifecycle management during tracking is seamlessly handled by the top-k strategy in temporal modelling, requiring no additional modifications. Specifics can be referred to in Algorithm 1. In our experiments (see Section IV-D3), we observe that the trained temporal model demonstrates excellent tracking characteristics without the need for fine-tuning with tracking constraints.

E. Multi-Modal Perception

The Sparse4D structure is flexible and can be extended to the LiDAR domain and the field of multi-modal fusion. As illustrated in Fig. 5, the process begins with the raw LiDAR point cloud, which is transformed into the BEV space through commonly used voxelization and post-processing operations [58]. Next, multi-scale features are extracted using a feature extractor. Similar to the operations in the camera domain, we also obtain instance features that represent objects along with their corresponding anchors. However, in this case, when projecting anchors onto the BEV features, only 2D projection coordinates are used to extract features at the corresponding BEV positions. These features are then aggregated to represent the LiDAR-specific characteristics of each object, facilitating instance feature extraction in the LiDAR domain. By simultaneously using feature extractors in both the image and LiDAR domains, we can obtain instance features from both sources. By merging the features of the same instance from these two domains, we achieve the final fused instance feature.

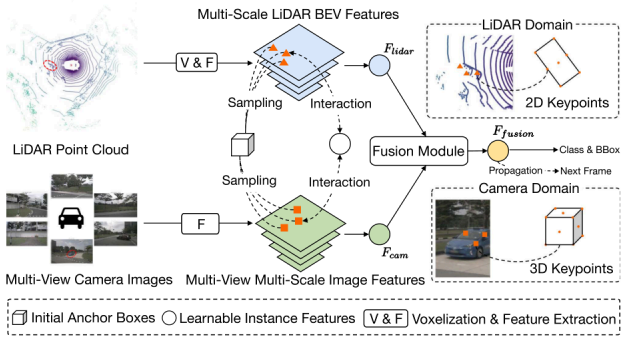


Fig. 5. Pipeline of Multi-modal perception, including LiDAR and camera.

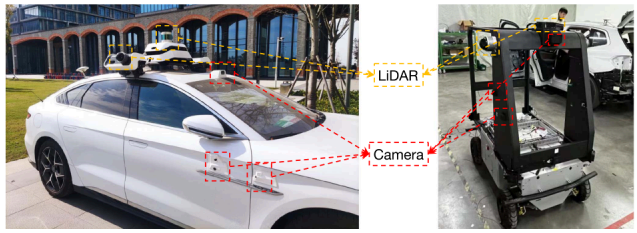


Fig. 6. Vehicles that are used for Sparse4D field runs.

F. Implementation Details

Sparse4D was validated not only by our extensive experiments in Section IV but also by implementation on two vehicles (Fig. 6): a modified passenger vehicle (BYD QIN) and an autonomous logistics vehicle. Both vehicles are equipped with our custom sensor suite (LiDAR and RGB cameras), NVIDIA Orin on-board computing unit, and Apollo open autonomous driving platform [59]. The vehicles were deployed in a dynamic urban road scenario (including downtown intersections, residential areas, and highway on-ramps) with five times of field runs (20 minutes per test).

The average latency (in milliseconds), mean Average Precision (mAP), and memory usage are compared qualitatively with baseline methods (SOLOFusion and StreamPETR) in Table II. These factors are crucial for reliable on-board deployment.

1) Temporal Instance Denoising: In 2D detection, incorporating a denoising task is an effective strategy for enhancing both model convergence stability and overall detection performance [60]. This paper extends the basic concept of 2D single-frame denoising to 3D temporal denoising for auxiliary

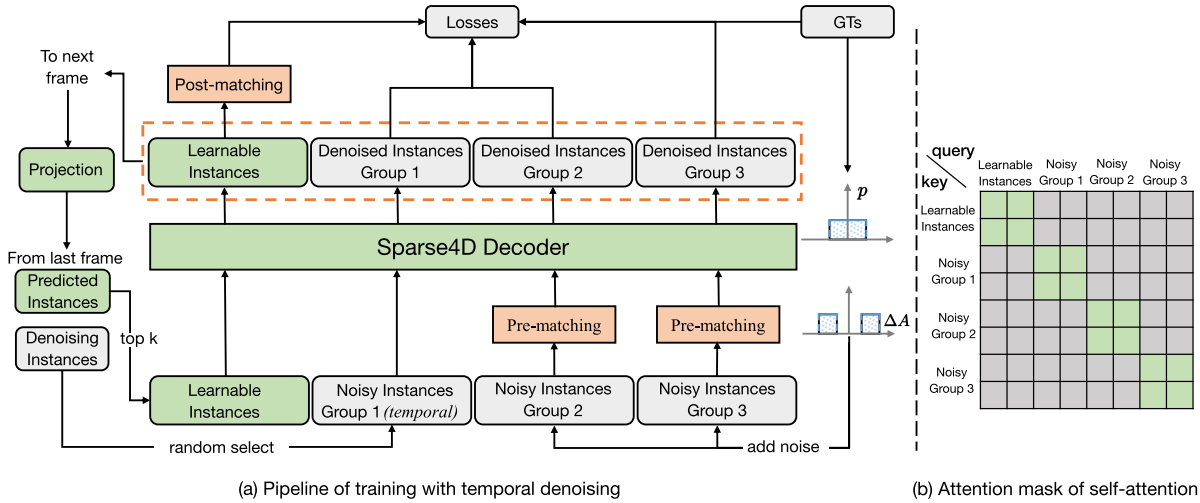


Fig. 7. Illustration of temporal instance denoising. (a) During the training phase, instances comprise two components: learnable and noisy. Noisy instances consist of both temporal and non-temporal elements. For noisy instances, we employ a pre-matching approach to allocate positive and negative samples by matching anchors with ground truth. In contrast, learnable instances are matched with predictions and ground truth. During the testing phase, only the green blocks in the diagram are retained. (b) An attention mask is employed to prevent feature propagation between groups, where grey indicates no attention between queries and keys, and green denotes the opposite. GT: Ground Truth. Best viewed in colour.

TABLE II
COMPARISON WITH SOLOFUSION [13] AND STREAMPETR [12] IN FIELD RUNS

Metric	Sparse4D	SOLOFusion	StreamPETR
mAP@0.5 (Dynamic Objects)	0.763	0.687	0.679
Inference Latency (ms)	128	231	153
Memory Usage (GB)	5.3	5.7	6.9

supervision. Within the Sparse4D framework, instances (referred to as queries) are separated into implicit instance features and explicit anchors. As shown in Fig. 7(a), during the training process, we initialize two sets of anchors. The first set consists of anchors that are uniformly distributed in the detection space, initialized using the k-means clustering method, and these anchors function as Learnable Instances. The second set of anchors is created by introducing noise to the ground truth data, namely Noisy Instances. Following the convention of (2), let A_{gt} denotes the ground truth:

$$A_{gt} = \{(x, y, z, w, l, h, yaw, v_{xyz})_i \mid i \in \mathbb{Z}_N\}. \quad (9)$$

where $\mathbb{Z}_X$ represents the set of integers between $[1, X]$. N denotes the number of ground truth.

During the training phase, we intentionally add noise with a specific distribution to the ground truth as the anchor A_{noise}, allowing the model to estimate the denoised ground truth:

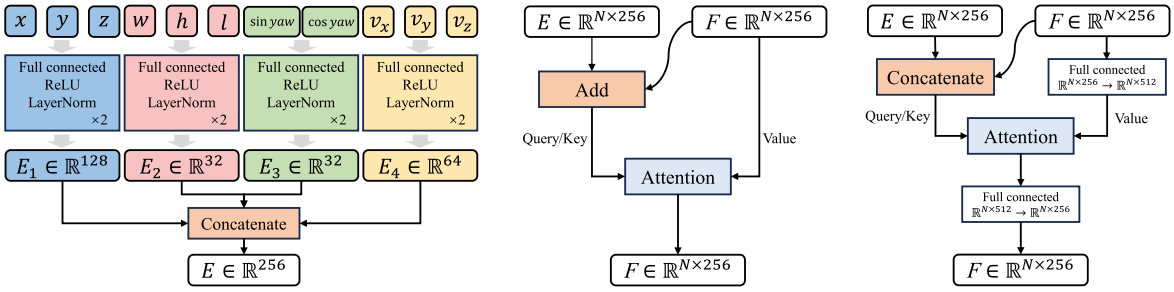
$$A_{noise} = \{A_i + \Delta A_{i,j,k} \mid i \in \mathbb{Z}_N, j \in \mathbb{Z}_M, k \in \mathbb{Z}_2\}. \quad (10)$$

where M represents the group number of noising instances. In this context, ΔA signifies random noise with probability density p, see Fig. 7(a), where $\Delta A_{i,j,1}$ and $\Delta A_{i,j,2}$ follow uniform random distributions within the range $(-x, x)$ and $(-2x, -x) \cup (x, 2x)$, respectively. It should be noted that DINO-DETR [60] categorizes samples generated by $\Delta A_{i,j,1}$ as positive and those from $\Delta A_{i,j,2}$ as negative. However, there is a potential risk of misassignment, as samples from $\Delta A_{i,j,2}$ may be closer to the ground truth. To eliminate any ambiguity, we employ bipartite graph matching for each group of A_{noise} and A_{gt} to determine which samples are positive and which are negative accurately.

Furthermore, we extend the aforementioned single-frame noisy instances through temporal propagation to better align with the sparse recurrent training process. During each frame's training, we randomly select M' $(M' \leqslant M)$ groups from the noisy instances to project onto the next frame. In Section IV, we empirically set $M = 5$ and $M' = 4$ based on our preliminary experiments [50]. The temporal propagation strategy aligns with that of non-noisy instances—anchors undergo ego pose and velocity compensation, while instance features serve as direct initializations for the features of the subsequent frame. It is important to note that we maintain the mutual independence of each group of instances, and no feature interaction occurs between noisy instances and normal instances. This is different from DN-DETR [61], as shown in Fig. 7(b). This approach ensures that within each group, a ground truth is matched to at most one positive sample, effectively avoiding any potential ambiguity.

2) Decoupled Attention: To address the issue of feature confusion in attention mechanisms, we propose straightforward enhancements to the anchor encoder, self-attention, and temporal cross-attention. As illustrated in Fig. 8, our design principle focuses on combining features from different modalities in a concatenated manner rather than employing an additive approach. There are notable differences compared to Conditional DETR [62]. First, as shown in Fig. 8(a) and (b), we enhance the attention ($E \in \mathbb{R}^{256}, F \in \mathbb{R}^{N \times 256}$) between queries instead of focusing solely on the cross-attention between the queries and the image features; the cross-attention still employs deformable aggregation from Sparse4D. Additionally, rather than concatenating the position embedding and query features at the single-head attention level, we implement modifications externally at the multi-head attention level (Fig. 8(c)), which provides the neural network with increased flexibility.

Thus, the contribution of Sparse4D lies not in individual components alone, but in the coherent integration of sparse 4D instance modelling, deformable multi-view aggregation,



(a) Anchor encoder (b) Vanilla Attention (c) Decoupled Attention

Fig. 8. Architecture of the anchor encoder and two instance-based attention. We independently conduct high-dimensional feature encoding on multiple components of the anchor in (a) and subsequently concatenate them in (b) and (c). This strategy leads to lower computational and parameter overhead. E and F represent anchor embedding and instance feature, respectively. Best viewed in colour.

long-term recurrent propagation, and temporal denoising into a single, efficient framework. To the best of our knowledge, Sparse4D is the first method to demonstrate that a fully sparse, instance-centric 4D representation can achieve competitive or superior performance to dense BEV-based approaches while being more scalable in time.

3) Dense Depth Supervision: In our preliminary experiments [49], we found that sparse-based methods struggled with convergence capability and speed during the early stages of training. To address this issue, we introduced multi-scale dense depth estimation, using point clouds as supervision. During inference, this sub-network will not be activated. For each scale of the feature map, we apply a 1×1 convolution to generate depth values at a pre-defined equivalent focal length. These depth values are then adjusted by multiplying them by the ratio of the camera's actual focal length to the equivalent focal length. The loss function used for dense depth supervision employs the standard L1 loss, as described in Section III-F4.

4) Loss Function and Quality Estimation: We sample video clips with T frames to train the detector end to end. The time interval between consecutive frames is randomly sampled in $\{d_t, 2d_t\}, (d_t \approx 0.5)$. Following DETR3D [10], the Hungarian algorithm is used to match each ground truth with one predicted value. The loss includes four parts: classification loss L_{cls}, bounding box regression loss L_{box}, depth estimation loss L_{depth}, and quality loss L_{qua}:

$$L = \lambda_1 L_{cls} + \lambda_2 L_{box} + \lambda_3 L_{depth} + \lambda_4 L_{qua}. \tag{11}$$

where $\lambda_1, \lambda_2, \lambda_3, \lambda_4$ are weight terms to balance the gradient of four parts. We adopt focal loss [63] for L_{cls}, L1 loss for L_{box}, binary cross entropy loss L_{depth}. In Section IV, we empirically set $\lambda = 1$ for all weight terms.

It is worth discussing whether to take L_{qua} into account. Existing sparse-based methods mainly estimate classification confidence for positive and negative samples to assess their alignment with the ground truth [10], [34], [46]. The primary optimization goal is to maximize the classification confidence of all positive samples. However, there is considerable variation in the matching quality among different positive samples. As a result, classification confidence is not an ideal metric for evaluating the quality of predicted bounding boxes. To help the network better understand the quality of positive samples, thereby accelerating convergence and rationalizing the prediction ranking, we introduce the task of prediction quality estimation. Specifically, for the 3D detection task, we define two quality metrics: centerness C and yawness Y. The centerness C aims to estimate the location error between the predicted and ground truth boxes:

$$C = \exp(-\|[x, y, z]_{pred} - [x, y, z]_{gt}\|_2). \tag{12}$$

where $[x, y, z]$ denotes the location of an object in 3D space. Similarly, yawness Y presents the rotation error between the predicted and ground truth boxes:

$$Y = [\sin yaw, \cos yaw]_{pred} \cdot [\sin yaw, \cos yaw]_{gt}. \tag{13}$$

where all symbols are consistent with the anchor in (2). While network outputs classification confidence L_{cls}, it also estimates centerness C and yawness Y via L_{qua}:

$$L_{qua} = \lambda_5 \mathbf{Foc}(Y_{pred}, \{Y|Y > 0\}) + \lambda_6 \mathbf{CE}(C_{pred}, C). \tag{14}$$

where loss functions of Y and C are defined as focal loss $\mathbf{Focal}(\cdot)$ [63] and cross-entropy loss $\mathbf{CE}(\cdot)$, respectively. Here, we only select positive samples of Y for the calculation. Similar to (11), λ_5 and λ_6 are empirically set to 1 based on our preliminary experiments [48], [49], [50] on nuScenes [11].

IV. Experiments

First, we introduce the dataset and the metrics used for evaluation. We also assess the proposed Sparse4D through a series of ablation studies, alongside quantitative and qualitative comparisons. It is important to note that Sparse4D has been implemented in various vehicles for field testing. Therefore, we will also be discussing the long-tail cases in the final section.

A. Dataset and Metrics

Dataset: We utilize the nuScenes dataset [11], which contains 1,000 scenes. The dataset is divided into training, validation, and testing sets with 700, 150, and 150 scenes, respectively. Each scene consists of a 20-second video clip recorded at two frames per second (FPS) and includes six viewpoint images. In addition to 3D bounding box labels, the dataset provides information on vehicle motion states and camera parameters.

Metrics: For detection, we adopt a comprehensive approach that considers several metrics: mean Average Precision (mAP), mean Average Error of Translation (mATE), mean Average Scale Error (mASE), mean Average Orientation Error (mAOE), mean Average Velocity Error (mAVE), mean Average Attribute Error (mAAE), and the nuScenes Detection Score (NDS). The NDS is a weighted average of the other metrics. In terms of tracking model evaluation, key metrics include Average Multi-Object

Algorithm 1: Tracking Pipeline of Sparse4D.

Input:
1) Sensor Data D,
2) Temporal Instances $I_t = \{(c, a, \mathrm{id})_i | i \in \mathbb{Z}_{N_t}\}$,
3) Current Instances $I_{cur} = \{a_i | i \in \mathbb{Z}_{N_{cur}}\}$. The terms c, a and id correspond to confidence, anchor box, and object ID, respectively, where the id may be empty.
Output:
1) Perception Results $R = \{(c, a, \mathrm{id})_i | i \in \mathbb{Z}_{N_r}\}$,
2) Updated $I'_t = \{(c, a, \mathrm{id})_i | i \in \mathbb{Z}_{N_t}\}$.

Define the confidence threshold T.
Define the confidence decay scale S.
Initialize $R = \{\}$.
Forward the model $R_{det} = \mathbf{Model}(D, I_t, I_{cur}) = \{(c', a')_i | i \in \mathbb{Z}_{N_t+N_{cur}}\}$.
for i ***from*** 1 **to** $N_t + N_{cur}$ ***do***
  if $c'_i \geq T$ **then**
    if $i > N_t$ *or* id_i *is empty* **then**
      Generate the new id_i.
    Put $(c'_i, a'_i, \mathrm{id}_i)$ into R.
  if $i \leq N_t$ **then**
    Update $c'_i = \max(c'_i, c_i \times S)$.
Select N_t instances with the highest confidence c' from the R_{det} to serve as I'_t.
Return R and I'_t.

Tracking Accuracy (AMOTA), Average Multi-Object Tracking Precision (AMOTP), Recall, and ID Switch (IDS). For a detailed understanding of these metrics, please refer to [11], [64].

B. Model and Training Setting

Model: To balance both efficiency and effectiveness, we have implemented a six-layer decoder consisting of 900 instances and $N_t = 600$ temporal instances, with an embedding dimension of 256. The model also includes seven fixed keypoints and six learnable keypoints. In Algorithm 1, the parameters T and S are set to 0.25 and 0.6, respectively. The number of groups for denoising instances, denoted as M, is 5. Among these, 4 groups are randomly selected as temporal denoising instances.

Training: We train the model for 100 epochs using the AdamW optimizer, without the need for class-balanced grouping and sampling (CBGS) [65], and no fine-tuning is required for the tracking task. We also employ a sequential iteration approach for training. Each training step processes input data from a single frame along with instances cached from previous frames. The training duration and GPU memory consumption of the temporal model are comparable to those of the single-frame model, enabling efficient training of the temporal model.

C. Ablation Study

We conducted a set of ablation studies to investigate the effectiveness of our proposed efficient deformable aggregation (EDA) module in Section III-B1, the tracking algorithm in Section III-D and other implementation details introduced in Section III. In all our ablation experiments, we used ResNet50 (pre-trained on ImageNet-1 K), with the input image size set to 256×704.

1) Efficient Deformable Aggregation: EDA has a significant impact on GPU memory usage and inference speed, as illustrated in Table IV. During the training phase, when the batch size is set to 1, GPU memory consumption decreases from 5705 MB to 2853 MB, representing a 50% reduction. Moreover, the maximum allowable batch size increases from 3 to 8. This results in the total training time for a complete experiment being reduced from 23.5 hours to 14.5 hours. These improvements lower the training barrier and enhance the efficiency of Sparse4D.

In the inference phase, EDA boosts the model's frames per second (FPS) from 13.1 to 19.8, marking an impressive improvement of approximately 51%. Additionally, EDA reduces GPU memory usage during inference by 48%, which decreases both the cost and complexity of deploying Sparse4D.

2) Tracking Algorithm: To demonstrate the effectiveness of our end-to-end tracking model, we replaced Algorithm 1 with a post-processing-based tracking method which utilizes a simple tracking-by-detection approach with cascading post-processing modules called SimpleTrack [66], while using the same detector. The comparison of metrics is presented in Table V. Algorithm 1 outperforms SimpleTrack across all metrics, showing a significant advantage in the key metric of ID switches. To confirm this observation, we also implement and compare the performance of CAMO-MOT [67].

Specifically, while keeping the prediction pipeline, track lifecycle management, and redundancy handling unchanged, we introduce an independent CAMO-MOT association branch that coexists with the original associator and adopts different matching strategies for key and non-key frames. In constructing the association cost, CAMO-MOT jointly considers geometric consistency (IoU/GIoU-based overlap) and motion consistency (using Euclidean or Mahalanobis distance), and further incorporates confidence priors and substantial cross-class penalties to prioritize high-confidence detections and reduce class confusion. To better model track reliability, we maintain state_score, hits, and age_since_update for each tracklet, strengthening them upon successful matches and softly decaying them otherwise, and use these states as normalization factors to adjust association weights. In addition, we integrate the CAMO-MOT gating mechanism after Hungarian matching to perform secondary filtering, applying different gate thresholds for key and non-key frames to achieve a better balance between matching precision and stability. It is clear that Algorithm 1 demonstrates superior overall performance in most metrics. It suggests that our model successfully achieves data association between frames by utilizing higher-dimensional features, rather than relying solely on structured information.

3) Comprehensive Modelling: Table III presents a quantitative overview of the effectiveness of each modelling and implementation approach proposed in Sparse4D. For example, the camera parameter encoding explicitly integrates camera parameters into the network. The metrics indicate that removing this module results in a decrease of approximately 7.5 in mAP and 12.5 in mAOE. In theory, estimating the orientation of an object relies not only on its image features but also on knowledge of its position in the coordinate system, along with the camera's intrinsic and extrinsic parameters. Consequently, the explicit encoding of camera parameters can enhance the accuracy of orientation estimation.

TABLE III
COMPREHENSIVE ABLATION EXPERIMENTS. IN THE LAST ROW, THE GREEN FONT INDICATES THE AMOUNT OF IMPROVEMENT IN THE METRIC.

Setting	mAP↑	mATE↓	mASE↓	mAOE↓	mAVE↓	mAAE↓	NDS↑	AMOTA↑	AMOTP↓
base model	0.323	0.725	0.277	0.597	0.832	0.209	0.397	-	-
+ dense depth supervision	0.341	0.705	0.276	0.587	0.820	0.217	0.410	-	-
+ multi-frame layer	0.384	0.621	0.271	0.524	0.281	0.180	0.504	0.346	1.301
+ temporal modelling	0.416	0.582	0.271	0.462	0.299	0.182	0.529	0.396	1.262
+ camera parameter encoding	0.439	0.598	0.270	0.475	0.282	0.179	0.539	0.414	1.268
+ single-frame denoising	0.447	0.586	0.267	0.455	0.257	0.187	0.548	0.445	1.230
+ decoupled attention	0.458	0.599	0.268	0.481	0.238	0.192	0.551	0.472	1.195
+ temporal denoising	0.462	0.581	0.269	0.454	0.246	0.189	0.557	0.457	1.191
+ centerness	0.463	0.563	0.265	0.517	0.221	0.208	0.554	0.466	1.180
+ yawness	0.469	0.553	0.274	0.476	0.227	0.200	0.561	0.490	1.164
= Sparse4D	**+0.128**	**−0.152**	**+0.002**	**+0.111**	**−0.593**	**+0.017**	**+0.151**	**+0.094**	**−0.098**

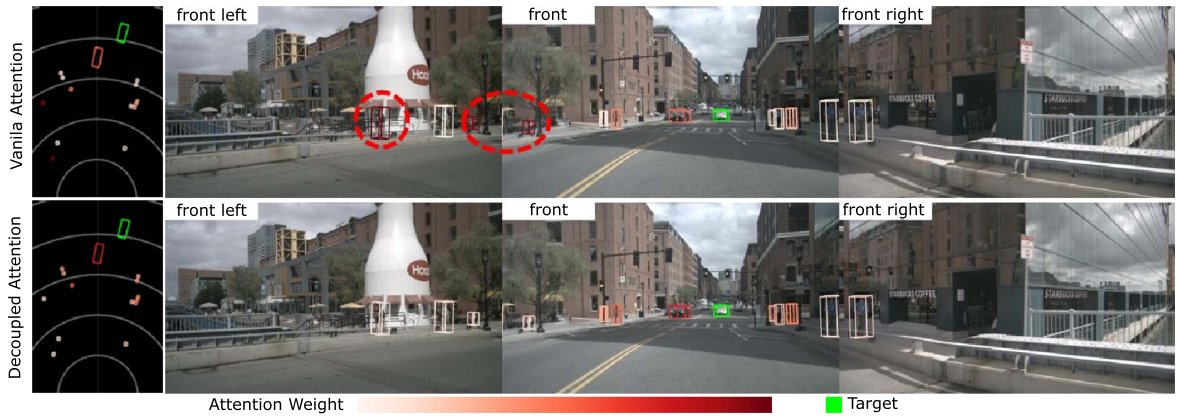


Fig. 9. Visualizing attention weights in instance self-attention: (1) The first row shows the attention weights in vanilla self-attention, where pedestrians (in red circles) display unintended similarities with the target vehicle (in green box). (2) The second row illustrates the attention weights of the decoupled attention, which effectively addresses this issue. Best viewed in colour.

TABLE IV
ABLATION ON EFFICIENT DEFORMABLE AGGREGATION (EDA). THIS EXPERIMENT WAS CONDUCTED USING RTX 3090 GPU WITH 24 GB MEMORY. FOR MEASUREMENTS OF TRAINING GPU MEMORY (MB), INFER GPU MEMORY (MB), AND INFER FPS, WE USED A BATCH SIZE OF 1. THE TRAINING TIME (HOUR) REPRESENTS THE TIME TAKEN TO TRAIN THE MODEL FOR 100 EPOCHS USING THE MAXIMUM BATCH SIZE ON 8 RTX 3090 GPUS.

EDA	Training			Inference	
	GPU Memory	Max Batch Size	Time	GPU Memory	FPS
	5705	3	23.5	964	13.1
✓	2853	8	14.5	469	19.8

TABLE V
ABLATION ON ALGORITHM 1 BY COMPARING OUR PROPOSED END-TO-END TRACKER TO SIMPLETRACK [66] AND CAMO-MOT [67] USING THE SAME DETECTOR

Method	AMOTA↑	AMOTP↓	Recall↑	IDS↓	MOTA↑	MOTP↓
SimpleTrack	0.486	1.181	0.559	575	0.426	0.662
CAMO-MOT	0.438	**1.082**	**0.575**	680	0.392	0.667
Algorithm 1	**0.490**	1.164	0.574	**430**	**0.436**	**0.655**

The decoupled attention mechanism is designed to improve the structure of the instance self-attention and temporal cross-attention modules in Sparse4D while minimizing feature interference during the computation of attention weights. As illustrated in Fig 9, when the anchor embedding and instance feature are combined for attention calculation, there are instances of outlier values in the resulting attention weights. This distorts the inter-correlation among target features, preventing the aggregation of the correct features. By replacing addition with concatenation, we significantly reduce the occurrence of this issue. Dense depth supervision is primarily included to facilitate the training of Sparse4D. As shown in Table III, eliminating dense depth supervision leads to a noticeable drop in performance metrics, with mAP and NDS decreasing by 1.8 and 1.3, respectively. This decline is attributed to the occurrence of gradient collapse during the training process.

The temporal decoder consists of a single-frame layer and five multi-frame layers. The single-frame layer is primarily responsible for selecting newly generated objects. If we change all layers in the decoder to multi-frame layers, the number of anchors allocated for handling new objects will decrease while still maintaining the total predetermined quantity of anchors. This reduction in anchors for new objects results in a decline in detection performance, as illustrated in Table III (aka. from + temporal modelling to + multi-frame layer), showing a decrease of 3.2 in mAP and 2.5 in NDS, respectively. Conversely, if we remove the multi-frame layers entirely and allow all layers in the decoder to receive input solely from the current frame, this will create a non-temporal fusion model. Comparing this non-temporal model to the temporal model demonstrates the significant impact of temporal fusion, resulting in an improvement of 9.3 in mAP and 13.2 in NDS.

4) Sensor Failure: To assess the robustness of Sparse4D in situations where a single sensor (such as a camera or LiDAR) fails, we conducted experiments to simulate three scenarios: 1) camera failure, 2) LiDAR failure, and 3) camera soiling (due to

TABLE VI
RESULTS OF 3D DETECTION ON NUSCENES VALIDATION DATASET. THE BEST PERFORMANCE FOR EACH METRIC IS HIGHLIGHTED IN BOLD.

Methods	Backbone	Image Size	mAP↑	mATE↓	mASE↓	mAOE↓	mAVE↓	mAAE↓	NDS↑	FPS↑
BEVPoolv2 [15]	ResNet50	256 × 704	0.406	0.572	0.275	0.463	0.275	0.188	0.526	16.6
BEVFormerv2 [68]	ResNet50	-	0.423	0.618	0.273	**0.413**	0.333	0.188	0.529	-
SOLOFusion [13]	ResNet50	256 × 704	0.427	0.567	0.274	0.511	0.252	0.181	0.534	11.4
VideoBEV [57]	ResNet50	256 × 704	0.422	0.564	0.276	0.440	0.286	0.198	0.535	-
StreamPETR [12]	ResNet50	256 × 704	0.432	0.609	**0.270**	0.445	0.279	0.189	0.537	**26.7**
Sparse4D	ResNet50	256 × 704	**0.469**	**0.553**	0.274	0.476	**0.227**	0.200	**0.561**	19.8
BEVDepth [17]	ResNet101	512 × 1408	0.412	0.565	0.266	0.358	0.331	0.190	0.535	-
SOLOFusion [13]	ResNet101	512 × 1408	0.483	0.503	0.264	0.381	0.246	0.207	0.582	-
StreamPETR [12]	ResNet101	512 × 1408	0.504	0.569	0.262	0.315	0.257	0.199	0.592	6.4
Far3D [51]	ResNet101	512 × 1408	0.510	0.551	0.258	0.372	0.238	0.195	0.594	-
RayDN [69]	ResNet101	512 × 1408	0.518	0.541	0.260	0.315	0.236	0.200	0.604	-
OPEN [70]	ResNet101	512 × 1408	0.516	0.528	0.266	0.312	0.222	**0.190**	0.606	-
GeoBEV [71]	ResNet101	512 × 1408	0.526	**0.458**	0.254	0.318	0.238	0.207	0.615	-
BEVNeXt [72]	ResNet101	512 × 1408	0.500	0.487	0.260	0.343	0.245	0.197	0.597	-
MemDistill [73]	ResNet101	512 × 1408	0.459	0.571	**0.227**	0.385	0.316	0.220	0.558	-
Sparse4D	ResNet101	512 × 1408	**0.537**	0.511	0.255	**0.306**	**0.194**	0.192	**0.623**	8.2

TABLE VII
RESULTS OF 3D DETECTION ON NUSCENES TEST DATASET. ALL VOVNET-99 IN THE TABLE ARE INITIALIZED WITH WEIGHTS FROM DD3D [74].

Methods	Backbone	Image Size	mAP↑	mATE↓	mASE↓	mAOE↓	mAVE↓	mAAE↓	NDS↑
HoP-BEVFormer [75]	VoVNet-99 [76]	640 × 1600	0.517	0.501	0.245	0.346	0.362	**0.105**	0.603
BEVStereo [16]	VovNet-99 [76]	640 × 1600	0.525	0.431	0.246	0.358	0.357	0.138	0.610
SOLOFusion [13]	ConvNeXt-B [77]	640 × 1600	0.540	0.453	0.257	0.376	0.276	0.148	0.619
BEVFormerv2 [68]	InternImage-B [78]	640 × 1600	0.540	0.488	0.251	0.335	0.302	0.122	0.620
VideoBEV [57]	ConvNeXt-B [77]	640 × 1600	0.554	0.457	0.249	0.381	0.266	0.132	0.629
BEVFormerv2 [68]	InternImage-XL [78]	640 × 1600	0.556	0.456	0.248	0.317	0.293	0.123	0.634
StreamPETR [12]	VoVNet-99 [76]	640 × 1600	0.550	0.479	0.239	0.317	0.241	0.119	0.636
RayDN [69]	VoVNet-99 [76]	640 × 1600	0.565	0.461	0.241	0.322	0.239	0.114	0.645
OPEN [71]	VoVNet-99 [76]	640 × 1600	0.567	0.456	0.244	0.325	0.240	0.129	0.644
GeoBEV [72]	VoVNet-99 [76]	640 × 1600	0.579	**0.369**	**0.234**	0.323	0.229	0.120	0.662
BEVNeXt [73]	VoVNet-99 [76]	640 × 1600	0.557	0.409	0.241	0.352	0.233	0.129	0.642
OcRFDet [79]	VoVNet-99 [76]	640 × 1600	0.572	0.421	0.253	0.331	0.248	0.130	0.648
Sparse4D	VoVNet-99 [76]	640 × 1600	0.570	0.412	0.236	0.312	0.210	0.117	0.656
Sparse4D	EVA02-large [80]	640 × 1600	**0.630**	0.379	0.235	**0.281**	**0.184**	0.127	**0.694**

TABLE VIII
COMPARISON BETWEEN SPARSE4D AND BEVFUSION [39] UNDER CAMERA AND LIDAR FAILS. C: ONLY CAMERA WORKS; C-SOILED: CAMERA SOILED; L: ONLY LIDAR WORKS; C+L: BOTH CAMERA AND LIDAR WORK.

Methods	Modality	mAP(val)↑	NDS(val)↑	mAP(test)↑	NDS(test)↑
BEVFusion	C	35.6	41.2	36.4	41.2
Sparse4D	C	46.9	56.1	47.2	58.1
BEVFusion	L	64.7	69.3	65.2	69.6
Sparse4D	L	66.6	71.1	66.4	71.9
BEVFusion	C+L	68.5	71.4	70.2	72.9
Sparse4D	C+L	71.3	73.6	71.4	74.4
BEVFusion	C-Soiled	34.4	40.5	-	-
Sparse4D	C-Soiled	45.8	55.8	-	-

factors such as dirt and fog). In each scenario, we evaluated the robustness of our proposed Sparse4D method against the widely used BEVFusion [39], using both qualitative and quantitative comparisons. As detailed in Table VIII, Sparse4D shows better and robuster performance than BEVFusion among all scenarios.

The primary reason Sparse4D excels is its ability to enhance robustness in sensor failure conditions through cross-view feature fusion, temporal modeling, and instance memory modules. This is particularly evident when handling occlusions (e.g., camera-soiled) and complex backgrounds. While BEVFusion [39] has strong feature extraction capabilities, it falls short in cross-view fusion and temporal modeling, which can result in missed detections and duplicate recognitions in intricate scenes. Consequently, Sparse4D demonstrates greater stability and accuracy when addressing complex occlusions, distant targets, and dynamic objects.

5) Camera Parameters: In Deformable Aggregation, the projection process from 3D key points P onto the feature map heavily relies on accurate camera calibration parameters (see (4)). Thus, we apply ablation studies on robustness against errors in the camera intrinsic and extrinsic parameters. Building on that, we compare the robustness of our proposed Sparse4D with AutoAlignV2 [81] under five different levels of error. To conduct this comparison, we simulate intrinsic and extrinsic camera errors, as well as joint errors. Specifically, we apply perturbations to focal length, principal point values, and extrinsic parameters by a random scaling factor in the range $[1-\epsilon, 1+\epsilon]$, where $\epsilon \in \{0.01, 0.02, 0.03, 0.04, 0.05\}$. As a result, both intrinsic and extrinsic perturbations are progressively increased across five noise levels (L1-L5).

As detailed in Fig. 12, Sparse4D consistently demonstrates stronger robustness than AutoAlignV2 under intrinsic, extrinsic, and joint calibration noise. We observe that Sparse4D exhibits slightly higher sensitivity in mASE as the noise level increases compared to AutoAlignV2. This behavior can be attributed to the instance-centric representation adopted by Sparse4D, in which scale estimation depends more directly on accurate geometric consistency across multi-view projections. Furthermore, when Sparse4D is additionally fine-tuned for five epochs under the 0.05 combined intrinsic-extrinsic noise setting and evaluated at

Fig. 10. Sparse 4D detection results observed under sunny, rainy, and night conditions. We assign different colors to different object types.

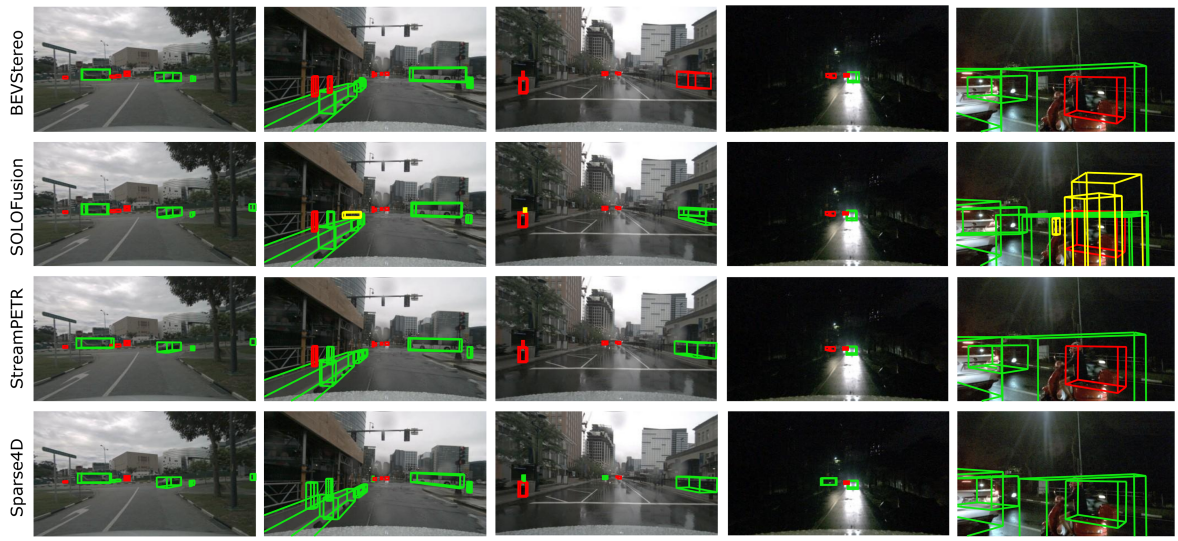


Fig. 11. Visual comparison of detection results from BEVStereo [16], SOLOFusion [13], StreamPETR [12] and our proposed Sparse4D. The correct (green), missing (red) and wrong (yellow) detections are marked. Best viewed in colour.

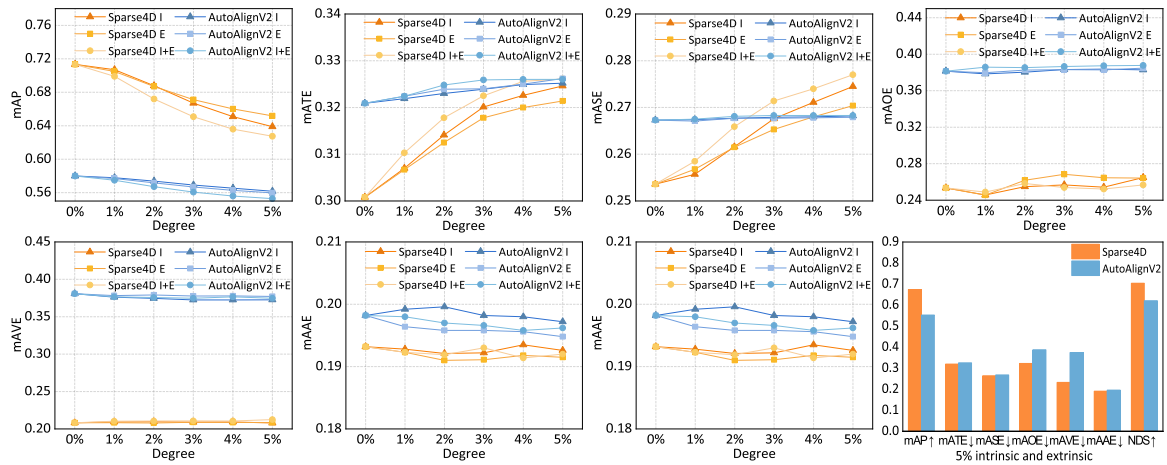


Fig. 12. Robustness comparison between Sparse4D and AutoAlignV2 under intrinsic, extrinsic, and combined calibration noise on the nuScenes validation set. The first seven subplots report performance trends under increasing noise levels from 0 to 0.05 across different metrics. For each method, *I*, *E*, and *I+E* denote intrinsic-only perturbation, extrinsic-only perturbation, and combined intrinsic–extrinsic perturbation, respectively. Sparse4D results are shown in orange with different shades and marker types indicating different noise settings, while AutoAlignV2 results are shown in blue using the same marker shapes for a fair comparison. The last subplot summarizes the performance evaluated under the 0.05 combined intrinsic extrinsic noise setting, where Sparse4D is additionally fine-tuned for five epochs with the same 0.05 combined noise and then tested under this noise level, and is compared against AutoAlignV2 evaluated under the identical 0.05 combined noise condition.

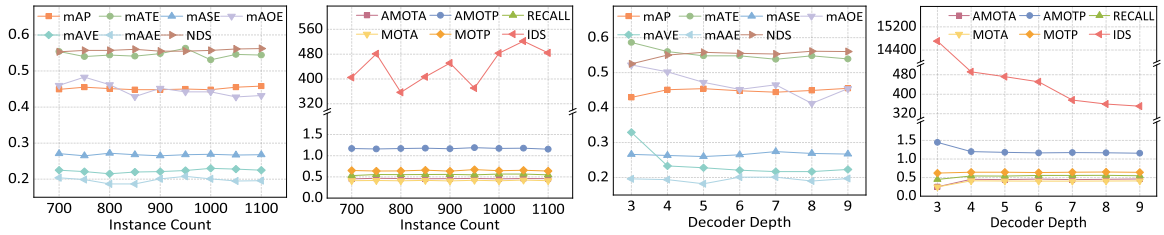


Fig. 13. Ablation studies on instance count and decoder depth.

the same noise level, its robustness is further improved. This result suggests that Sparse4D can effectively adapt to calibration perturbations through noise-aware training.

6) Decoder Depth and Instance Count: We fix the decoder depth at six and vary the instance count from 700 to 1100. In Fig. 13 (the first two sub-figures), we observe that both increasing and decreasing the number of instances result in minor fluctuations in specific metrics. However, the overall changes are limited and accompanied by inconsistent variations across different metrics. For example, using 850 instances leads to a slight improvement in NDS and MOTA, but metrics like mAOE and mASE show a decline. The 900-instance configuration proved to be the most promising, maintaining high levels across key metrics such as mAP, NDS, AMOTA, and IDS, without apparent weaknesses, and demonstrating the highest overall robustness.

We fix the instance count at 900 and vary the decoder depth from 3 to 9. As shown in Fig. 13 (the last two sub-figures), we notice that reducing the number of decoder layers to 3 or 4 results in a significant drop in performance, particularly a steep decline in IDS efficacy, which severely impacts tracking stability. On the other hand, increasing the depth to 7–9 layers yields only minimal gains in accuracy while substantially raising computational costs. Therefore, using six layers provides the optimal balance between performance improvement and computational efficiency.

In terms of efficiency-performance tradeoff, we find that the 900×6 configuration achieves consistently high levels in detection quality (mAP/NDS), geometric error (mATE/mAOE/mASE), and association stability (AMOTA/MOTA/IDS) without incurring higher computational complexity. While configurations with 850 or 800 instances show some advantages in specific metrics, they lack consistency, and the overall improvements are not statistically significant. Additionally, increasing the decoder beyond six layers results in higher computational costs without yielding practical performance gains. Based on the findings from this ablation study, we select the 900-instance configuration with a 6-layer decoder as the default setting. This choice ensures that the model delivers the most stable, reproducible, and generalizable performance across various detection and tracking tasks, aligning with the optimal point of the efficiency-performance tradeoff.

D. Comparison

We evaluate Sparse4D on 3D object detection and multi-object tracking tasks using various test sets, backbones, and multi-modal domains, and compare it with state-of-the-art methods.

1) 3D Detection on Validation Set: To effectively control variables, we conduct comprehensive comparative experiments on the validation dataset, as detailed in Table VI. In the first experimental setup, we utilize ResNet50 [82] as the backbone, initialized with parameters obtained from supervised training on the ImageNet-1k [55] dataset. The image size is set to 256×704. This configuration requires relatively lower GPU memory and training time, which helps facilitate experimental iterations. In this setup, Sparse4D improves the mAP and the NDS by 3.0% and 2.2%, respectively.

In the second experimental setup, we use ResNet101 as the backbone and increase the image size to 512×1408 to evaluate the model's performance with higher-resolution images. The backbone was pre-trained on the nuImages [11] dataset, a large-scale image dataset that uses the same sensor setup as the 3D nuScenes dataset. Sparse4D achieves state-of-the-art performance in this configuration, with mAP and NDS improving by 3.2% and 2.9%, respectively. Furthermore, our inference speed exceeds that of StreamPETR [12], with 8.2 compared to 6.4 FPS.

2) 3D Detection on Test Set: We evaluate the model's performance on the nuScenes test dataset, as indicated in Table VII. To ensure consistent configurations with most algorithms, we use VoVNet-99 [76] as the backbone, utilizing pre-trained weights from DD3D [74] and setting the image size to 640×1600. Compared with state-of-the-arts, Sparse4D achieves optimal performance in both metrics, with mAP and NDS improving by 1.3% and 1.8%, respectively. Notably, the distance error such as mATE of our sparse-based algorithm significantly exceeds that of the dense BEV-based algorithms. This improvement is primarily due to the stability in confidence ranking achieved through our quality estimation, resulting in a substantial enhancement in mATE. Later, we employ a larger backbone EVA02-large [80], our proposed Sparse4D continues to deliver the best performance across most metrics.

For the quantitative evaluation, Fig. 10 illustrates the detection results of Sparse4D under various weather conditions. It is clear that there are instances of incorrect and missing object detections, particularly in low-light conditions. However, Fig. 11 shows that the 3D bounding boxes generated by Sparse4D align more closely with the targets and demonstrate fewer missing objects compared to the actively used approaches listed in Table VII.

3) Multi-Object Tracking 3D: As shown in Table IX, Sparse4D significantly outperforms existing methods across all tracking metrics on the validation set, regardless of whether the methods are end-to-end or non-end-to-end. For instance, compared to the state-of-the-art solution DORT [84], Sparse4D achieves an AMOTA that is 6.6% higher (0.490 vs. 0.424) under the same configuration. When compared to the end-to-end solution DQTrack [85], our AMOTA improves by 16.0% (0.567 vs. 0.407), and the number of ID switches is reduced by 44.5% (557 vs. 1003). Sparse4D outperforms S2-Track [86] (even when S2-Track uses a larger VoVNet-99 backbone and high-resolution inputs of 640×1600) regarding AMOTA, MOTA, and Recall.

TABLE IX
RESULTS OF 3D MULTI-OBJECT TRACKING ON NUSCENES VAL DATASET. E2E: END-TO-END DETECTION AND TRACKING. SPARSE4D IS SAME AS IN TABLE VI AND VII.

Method	E2E	Backbone	Image size	AMOTA↑	AMOTP↓	IDS↓	Recall↑	MOTA↑	MOTP↓
QTrack [83]		ResNet50	256 × 704	0.347	1.347	944	0.426	0.426	0.722
DORT [84]		ResNet50	256 × 704	0.424	1.264	-	0.492	-	-
Sparse4D	✓	ResNet50	256 × 704	**0.490**	**1.164**	**430**	**0.574**	**0.436**	**0.660**
DQTrack [85]	✓	ResNet101	512 × 1408	0.407	1.318	1003	-	-	-
SRCN3D [20]		VoVNet-99	900 × 1600	0.439	1.280	-	0.545	-	-
QTrack [83]		VoVNet-99	640 × 1600	0.511	1.090	1144	0.585	0.465	-
S2-Track [86]	✓	VoVNet-99	640 x 1600	0.566	1.090	**174**	0.622	0.498	0.657
PF-Track [87]	✓	VoVNet-99	640 x 1600	0.479	1.227	181	0.590	0.435	-
ADA-Track-long [88]	✓	ResNet-101	900 x 1600	0.392	1.375	897	0.523	0.363	-
ADA-Track [88]	✓	VoVNet-99	640 x 1600	0.479	1.246	767	0.602	0.430	-
Sparse4D	✓	ResNet101	512 × 1408	**0.567**	**1.027**	557	**0.658**	**0.515**	**0.621**

TABLE X
RESULTS OF 3D MULTI-OBJECT TRACKING ON NUSCENES TEST DATASET. E2E: END-TO-END DETECTION AND TRACKING. QTRACK-STP: QTRACK [85] WITH STREAMPETR [12] AS DETECTOR. SPARSE4D IS SAME AS IN TABLE VI AND VII.

Method	E2E	Backbone	AMOTA↑	AMOTP↓	IDS↓	Recall↑	MOTAR↑	MOTA↑	MOTP↓
MUTR3D [32]	✓	ResNet101	0.270	1.494	6018	0.411	0.643	0.245	0.709
SRCN3D [20]		VoVNet-99	0.398	1.317	4090	0.538	0.702	0.359	0.709
DQTrack [85]	✓	VoVNet-99	0.523	1.096	1204	0.622	-	0.444	0.649
QTrack-StP [12], [83]		ConvNext-B	0.566	0.975	784	0.650	0.711	0.460	0.576
DORT [84]		ConvNext-B	0.576	0.951	774	0.634	0.771	0.484	0.536
S2-Track [86]	✓	VoVNet-99	0.608	0.925	963	**0.758**	-	0.547	0.559
PF-Track [87]	✓	VoVNet-99	0.434	1.252	**249**	0.538	-	0.378	-
ADA-Track [88]	✓	VoVNet-99	0.456	1.237	834	0.559	-	0.406	-
Sparse4D	✓	VoVNet-99	0.574	0.970	669	0.669	0.779	0.521	0.525
Sparse4D	✓	EVA02-Large	**0.643**	**0.820**	699	0.743	**0.799**	**0.593**	**0.481**

TABLE XI
MULTI-MODAL 3D DETECTION RESULTS ON THE NUSCENES VALIDATION AND TEST SETS. THE BEST RESULTS ARE MARKED IN BOLD. IT IS NOTEWORTHY THAT WE DO NOT APPLY ANY TEST TIME AUGMENTATION (TTA) DURING THE TESTING PHASE. L: LIDAR. C: CAMERA.

Methods	Modality	LiDAR Backbone	Camera Backbone	validation set		test set	
				NDS	mAP	NDS	mAP
FCOS3D [89]	C	-	ResNet-101	41.5	34.3	42.8	35.8
PETR [27]	C	-	ResNet-101	44.2	37.0	45.5	39.1
BEVFormer [14]	C	-	ResNet-101	51.7	41.6	53.5	44.5
PromptDet [90]	C	-	ResNet-101	56.9	43.3	-	-
Sparse4D	C	-	ResNet-101	**62.3**	**53.7**	**64.5**	**54.1**
CenterPoint [91]	L	VoxelNet	-	66.8	59.6	67.3	60.3
TransFusion-L [45]	L	VoxelNet	-	70.1	65.1	70.2	65.5
VoxelNeXt [92]	L	ResVoxelNet	-	68.7	63.5	70.0	64.5
FSD-V2 [93]	L	SparseUNet	-	70.4	64.7	71.7	66.2
CenterPoint-SAE [94]	L	SSDCM	-	62.5	53.3	-	-
DAFDeTr-L [95]	L	VoxelNet	-	68.9	63.1	69.1	63.4
HP-PV-RCNN [96]	L	Sparse Voxel 3D	-	70.3	65.1	-	-
AS-Det [97]	L	PointNet2AS	-	61.8	51.9	-	-
Sparse4D	L	ResNet-18	-	**71.1**	**66.6**	**71.9**	**66.4**
FUTR3D [44]	L+C	VoxelNet	ResNet-101	68.3	64.5	-	-
UVTR [98]	L+C	VoxelNet	ResNet-101	70.2	65.4	71.1	67.1
BEVFusion [39]	L+C	VoxelNet	Swin-T	71.4	68.5	72.9	70.2
TransFusion [45]	L+C	VoxelNet	ResNet-50	71.3	67.5	71.6	68.9
DeepInteraction [99]	L+C	VoxelNet	ResNet-50	72.6	69.9	73.4	70.8
CMT [46]	L+C	VoxelNet	ResNet-50	70.8	67.9	-	-
SparseFusion [47]	L+C	VoxelNet	ResNet-50	72.8	70.4	73.8	72.0
PromptDet [90]	L+C	Voxel Encoding + 3D Conv	ResNet-101	64.1	56.2	-	-
RoboFusion [100]	L+C	VoxelNet	SAM	72.1	69.9	72.0	69.9
PF3Det [101]	L+C	VoxelNet	ViT-L	70.4	66.9	-	-
DAFDeTr [95]	L+C	VoxelNet	ResNet-101	69.7	64.6	69.3	64.6
Sparse4D	L+C	ResNet-18	ResNet-50	**73.6**	**71.3**	**74.4**	**71.4**

Table X presents the evaluation results on the test set, the advantages of Sparse4D become even more evident. With the enhanced EVA02-Large backbone, Sparse4D improves its AMOTA from 0.574 to 0.643 and MOTA from 0.521 to 0.593. This demonstrates both the scalability of the framework and the benefits of stronger feature representations. Therefore, Sparse4D not only achieves competitive quantitative performance compared to specialized multi-object tracking frameworks but also shows greater practical value and engineering feasibility for real-world large-scale autonomous driving applications.

4) Multi-Modal 3D Detection: Table XI shows the performance of models that have been extended to the LiDAR and multi-modal domains using the nuScenes dataset. These are the advanced methods in LiDAR-only and LiDAR-camera fusion

TABLE XII
EXPERIMENTAL RESULTS OF FUTURE FRAMES AND LARGER BACKBONES, WHERE THE IMAGE SIZE IS SET TO 640 × 1600. THE HEADER "FF" REPRESENTS THE NUMBER OF FUTURE FRAMES UTILIZED. TRANSFUSION-L AND TRANSFUSION-CL DENOTE PURE LIDAR MODELS AND MULTI-MODAL (CAMERA + LIDAR) MODELS, RESPECTIVELY.

Method	Backbone	FF	mAP↑	mATE↓	mAOE↓	mAVE↓	NDS↑	AMOTA↑	AMOTP↓
TransFusion-L [45]	DLA34&VoxelNet	0	0.655	0.256	0.351	0.278	0.702	0.686	0.529
TransFusion-CL [45]	DLA34&VoxelNet	0	**0.689**	**0.259**	0.359	0.288	0.717	**0.718**	**0.551**
Sparse4D	ResNet101	0	0.570	0.411	0.303	0.202	0.658	0.571	0.995
Sparse4D	ResNet101	8	0.613	0.371	0.285	0.144	0.690	0.608	0.876
Sparse4D	EVA02-Large [80]	0	0.630	0.379	0.281	0.184	0.694	0.643	0.820
Sparse4D	EVA02-Large [80]	8	0.668	0.346	**0.279**	**0.142**	**0.719**	0.677	0.761

TABLE XIII
PERFORMANCE OF SPARSE4D ON DIFFERENT WEATHER CONDITIONS FROM THE NUSCENES VAL DATASET

Weather	mAP↑	mATE↓	mASE↓	mAOE↓	mAVE↓	mAAE↓	NDS↑	AMOTA↑	AMOTP↓	Recall↑	MOTA↑	MOTP↓
night	0.268	0.749	0.452	0.460	0.616	0.629	0.344	0.487	1.178	0.615	0.514	0.551
rain	0.518	0.482	0.284	0.561	0.191	0.124	0.595	0.504	1.054	0.633	0.504	0.576
other	0.461	0.552	0.259	0.431	0.215	0.223	0.563	0.490	1.139	0.589	0.440	0.674

technologies in recent years. Several of them, such as FSD-V2 [93], HP-PV-RCNN [96], and VoxelNeXt [92], have publicly demonstrated superior performance on the Waymo dataset [102]. The results indicate that models utilizing multi-modal information achieve better NDS and mAP scores, demonstrating the effectiveness and advantages of our proposed Sparse4D in the area of multi-modal fusion. Additionally, it is important to note that our method also surpasses traditional LiDAR-only techniques when applied in a pure LiDAR setting. This suggests that the query-based paradigm is both applicable and beneficial, even within the LiDAR domain.

E. Cloud-Based Performance Boost

With the ample computational power available in cloud-based systems, it is common to leverage larger computational resources to achieve optimal performance. To fully utilize the potential of Sparse4D, we implement two key measures: feature fusion with future frames and the use of a larger, better-pretrained backbone. The results are detailed in Table XII.

We adopt a multi-frame sampling approach to fuse features from future frames (FF). By including features from the subsequent eight frames (referred to as 2 FPS), we observe a significant improvement in model performance, evidenced by a 5.67% reduction in mAVE and a 3.23% increase in NDS. It should be noted that the main objectives of Table XII is to provide a fair quantitative comparison (FF = 0) with TransFusion [45], and it also demonstrates the scalability and offline upper bounds of Sparse4D when cloud or offline inference is allowed (FF > 0).

Thus, following the methodology implemented in StreamPETR [12], we experiment with using EVA02 [80] as the backbone. EVA02 has undergone thorough pretraining, which enhances its feature extraction capability and enriches semantic information. This leads to improved generalization and better model classification. In comparison to ResNet101, EVA02-Large achieves a 5.98% increase in mAP. By integrating both EVA02 and future frames, we achieve remarkable results on the nuScenes test dataset, including an mAP of 0.682, an NDS of 0.719, and an AMOTA of 0.677. These achievements even surpass certain metrics (e.g., NDS and mAVE), when compared to detection models that utilize LiDAR, such as TransFusion [45] (the top two rows in Table XII).

Fig. 14. Long-tail cases observed in our experiments. Correct, missing and wrong detections are marked in green, red, and yellow, respectively. The rightmost column illustrates the zoomed-in results of small targets.

F. Long-Tail Cases

Fig. 14 showcases several typical long-tail cases observed during our extensive experiments and field runs. In the sunny scenario (first row), some distant, occulted cars and pedestrians are missed, as indicated in red in the first image. In the second image, a guideboard is incorrectly identified as a truck (marked in yellow) due to its similar shape and texture to that of container trucks. Additionally, some overcrowded targets are overlooked (marked in red and zoomed in) in the third image because of light rays and occlusion. In the rainy scenario (second row), a greater number of targets are missed across most images, primarily due to low light, fog, and reflective conditions. Similarly, in the night scenario (third row), small targets are often missed because of low light and occlusion. Lastly, missed detections frequently occur in shadow scenarios (bottom row). For example, in the third image, the bodies of a car and a pedestrian are partially obscured by shadows (see the zoomed-in), leading to additional missed detections.

To quantitatively evaluate the performance of Sparse4D under different weather and light conditions, we first categorize scenes using keywords from the official nuScenes description, including night and rain scenes. It is important to note that scenes can overlap (i.e., a scene can be both night and rain). All other scenes, which do not include either "night" or "rain," are grouped as "other." Since this categorization is done at the scene level and

TABLE XIV
PERFORMANCE OF SPARSE4D UNDER VARIOUS LIGHTING CONDITIONS USING THE NUSCENES VALIDATION DATASET

Light	mAP↑	mATE↓	mASE↓	mAOE↓	mAVE↓	mAAE↓	NDS↑
dark	0.396	0.587	0.254	0.510	0.341	0.230	0.506
mid	0.472	0.523	0.266	0.469	0.231	0.173	0.570
bright	0.425	0.647	0.269	0.422	0.310	0.253	0.522

preserves complete sequences, it ensures temporal continuity, allowing for stable comparisons in both detection and tracking. As detailed in Table XIII, Sparse4D demonstrates strong performance across various weather conditions. In scenarios of rain and others, it achieves notable detection metrics with NDS scores of 0.595 and 0.563, and mAP scores of 0.518 and 0.461, respectively. It also maintains stable tracking quality, with AMOTA values around 0.49 to 0.50 and similar recall rates. The consistently strong results in the rain subset reveal that Sparse4D is resilient to visual degradation caused by rain, benefiting from temporal aggregation and multi-view fusion to ensure reliable cues. The night subset poses the greatest challenge and naturally decreases detection accuracy. Nonetheless, Sparse4D still achieves an NDS of 0.344 and an AMOTA of 0.487, demonstrating that temporal associations remain effective, even under low-visibility conditions.

We evaluate lighting robustness on the nuScenes validation set using sample-level brightness bucketing. For each sample, we compute the mean Y-channel luminance across six camera views and divide samples by quantiles into dark (lowest 20%), mid (60%), and bright (highest 20%) groups. Since bucketing fragments scenes and disrupts tracking continuity, we report detection metrics only and exclude IDS for cross-subset comparisons due to unstable sequence lengths. Table XIV demonstrates that Sparse4D maintains strong detection performance across all lighting conditions. It achieves the highest overall accuracy in mid-light conditions, with a NDS of 0.570 and an mAP of 0.472. Additionally, it remains competitive in both darker and brighter environments, recording a NDS of 0.506 and mAP of 0.396 in dark conditions, as well as a NDS of 0.522 and mAP of 0.425 in bright conditions. This indicates that Sparse4D is not overly sensitive to moderate variations in exposure and consistently performs reliably across different lighting levels.

V. CONCLUSION

This paper introduces Sparse4D, an end-to-end sparse perception framework for 3D object detection in autonomous driving. It features (1) a deformable aggregation module for high-quality sparse feature fusion, (2) a recurrent instance feature propagation structure for efficient long-term temporal modeling, and (3) strong extensibility to multi-object tracking, multi-modal perception, and downstream tasks such as mapping and occupancy. The code is available on https://github.com/HorizonRobotics/Sparse4D.

REFERENCES

[1] L. Chen, P. Wu, K. Chitta, B. Jaeger, A. Geiger, and H. Li, "End-to-end autonomous driving: Challenges and frontiers," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 10164–10183, Dec. 2024.

[2] M. Liang et al., "PnPNet: End-to-end perception and prediction with tracking in the loop," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 11553–11562.

[3] Y. Hu et al., "Planning-oriented autonomous driving," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 17853–17862.

[4] B. Jiang et al., "VAD: Vectorized scene representation for efficient autonomous driving," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 8340–8350.

[5] X. Weng et al., "Para-drive: Parallelized architecture for real-time autonomous driving," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 15449–15458.

[6] C. Yang et al., "MLife: A lite framework for machine learning lifecycle initialization," *Mach. Learn.*, vol. 110, pp. 2993–3013, 2021.

[7] Y. Ma et al., "Vision-centric BEV perception: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 10978–10997, Dec. 2024.

[8] L. Guan and X. Yuan, "Instance segmentation model evaluation and rapid deployment for autonomous driving using domain differences," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 4, pp. 4050–4059, Apr. 2023.

[9] S. Xie et al., "Benchmarking and improving bird's eye view perception robustness in autonomous driving," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 5, pp. 1–17, May 2025.

[10] Y. Wang et al., "DETR3D: 3D object detection from multi-view images via 3D-to-2D queries," in *Proc. Conf. Robot Learn.*, 2022, pp. 180–191.

[11] H. Caesar et al., "nuScenes: A multimodal dataset for autonomous driving," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 11621–11631.

[12] S. Wang et al., "Exploring object-centric temporal modeling for efficient multi-view 3D object detection," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 3621–3631.

[13] J. Park et al., "Time will tell: New outlooks and a baseline for temporal multi-view 3D object detection," in *Proc. Int. Conf. Learn. Representations*, 2023, pp. 1–25.

[14] Z. Li et al., "BEVFormer: Learning bird's-eye-view representation from LiDAR-camera via spatiotemporal transformers," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 3, pp. 2020–2036, Mar. 2025.

[15] J. Huang and G. Huang, "BEVpoolv2: A cutting-edge implementation of BEVDet toward deployment," 2022, *arXiv:2211.17111*.

[16] Y. Li et al., "BEVStereo: Enhancing depth estimation in multi-view 3D object detection with temporal stereo," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 1486–1494.

[17] Y. Li et al., "BEVDepth: Acquisition of reliable depth for multi-view 3D object detection," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 1477–1485.

[18] Y. Liu et al., "PETRv2: A unified framework for 3D perception from multi-camera images," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 3262–3272.

[19] Z. Chen et al., "Graph-DETR3D: Rethinking overlapping regions for multi-view 3D object detection," in *Proc. ACM Int. Conf. Multimedia*, 2022, pp. 5999–6008.

[20] Y. Shi et al., "SRCN3D: Sparse R-CNN 3D surround-view camera object detection and tracking for autonomous driving," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops*, 2023, pp. 1–10.

[21] Z. Sun et al., "A survey of multiple pedestrian tracking based on tracking-by-detection framework," *IEEE Trans. Circuits Systems Video Technol.*, vol. 31, no. 5, pp. 1819–1833, May 2021.

[22] Q. Xu et al., "End-to-end joint multi-object detection and tracking for intelligent transportation systems," *Chin. J. Mech. Eng.*, vol. 36, no. 1, 2023, Art. no. 138.

[23] P. Sun et al., "TransTrack: Multiple object tracking with transformer," 2020, *arXiv:2012.15460*.

[24] T. Meinhardt et al., "TrackFormer: Multi-object tracking with transformers," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 8844–8854.

[25] J. Huang et al., "Detecting as labeling: Rethinking LiDAR-camera fusion in 3D object detection," in *Proc. Eur. Conf. Comput. Vis.*, 2024, pp. 439–455.

[26] J. Philion and S. Fidler, "Lift, splat, shoot: Encoding images from arbitrary camera rigs by implicitly unprojecting to 3D," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 194–210.

[27] Y. Liu et al., "PETR: Position embedding transformation for multi-view 3D object detection," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 531–548.

[28] K. He et al., "Mask R-CNN," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 2961–2969.

[29] N. Carion et al., "End-to-end object detection with transformers," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 213–229.

[30] F. Zeng et al., "MOTR: End-to-end multiple-object tracking with transformer," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 659–675.

[31] Y. Zhang et al., "MOTRv2: Bootstrapping end-to-end multi-object tracking by pretrained object detectors," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 22056–22065.

[32] T. Zhang et al., "Mutr3D: A multi-camera tracking framework via 3D-to-2D queries," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 4537–4546.

[33] A. H. Lang et al., "PointPillars: Fast encoders for object detection from point clouds," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 12697–12705.

[34] V. R. Kumar et al., "OmniDet: Surround view cameras based multi-task visual perception network for autonomous driving," *IEEE Robot. Automat. Lett.*, vol. 6, no. 2, pp. 2830–2837, Apr. 2021.

[35] H. Wang et al., "DSVT: Dynamic sparse voxel transformer with rotated sets," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 13520–13529.

[36] Z. Liu et al., "SEED: A simple and effective 3D DETR in point clouds," in *Proc. Eur. Conf. Comput. Vis.*, 2024, pp. 110–126.

[37] G. Zhang et al., "HEDNet: A hierarchical encoder-decoder network for 3D object detection in point clouds," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 53076–53089.

[38] Z. Liu et al., "LION: Linear group RNN for 3D object detection in point clouds," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 13601–13626.

[39] Z. Liu et al., "BEVFusion: Multi-task multi-sensor fusion with unified bird's-eye view representation," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2023, pp. 2774–2781.

[40] X. Chen et al., "Multi-view 3D object detection network for autonomous driving," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 1907–1915.

[41] X. Zhang et al., "Multi-modal fusion technology based on vehicle information: A survey," *IEEE Trans. Intell. Veh.*, vol. 8, no. 6, pp. 3605–3619, Jun. 2023.

[42] Z. Yang, N. Song, W. Li, X. Zhu, L. Zhang, and P. H. S. Torr, "Deep-interaction : Multi-modality interaction for autonomous driving," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, pp. 6749–6763, 2025.

[43] J. Yin et al., "Is-fusion: Instance-scene collaborative fusion for multi-modal 3D object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 14905–14915.

[44] X. Chen, T. Zhang, T. Zhang, Y. Wang, Y. Wang, and H. Zhao, "FUTR3D: A unified sensor fusion framework for 3D detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 172–181.

[45] X. Bai et al., "Transfusion: Robust LiDAR-camera fusion for 3D object detection with transformers," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 1090–1099.

[46] J. Yan et al., "Cross modal transformer: Towards fast and robust 3D object detection," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 18268–18278.

[47] Y. Xie et al., "SparseFusion: Fusing multi-modal sparse representations for multi-sensor 3D object detection," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 17591–17602.

[48] X. Lin, T. Lin, Z. Pei, L. Huang, and Z. Su, "Sparse4D: Multi-view 3D object detection with sparse spatial-temporal fusion," 2022, *arXiv:2211.10581*.

[49] X. Lin, T. Lin, Z. Pei, L. Huang, and Z. Su, "Sparse4D V2: Recurrent temporal fusion with sparse model," 2023, *arXiv:2305.14018*.

[50] X. Lin, Z. Pei, T. Lin, L. Huang, and Z. Su, "Sparse4D V3: Advancing end-to-end 3D detection and tracking," 2023, *arXiv:2311.11722*.

[51] X. Jiang et al., "Far3D: Expanding the horizon for surround-view 3D object detection," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 2561–2569.

[52] Z. Li et al., "FB-BEV: BEV representation from forward-backward view transformations," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 6919–6928.

[53] A. Waswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.

[54] X. Zhu et al., "Deformable DETR: Deformable transformers for end-to-end object detection," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–16.

[55] A. Krizhevsky et al., "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, 2017.

[56] W.-H. Chen et al., "CMOS-integrated memristive non-volatile computing-in-memory for AI edge processors," *Nature Electron.*, vol. 2, no. 9, pp. 420–428, 2019.

[57] C. Han et al., "Exploring recurrent long-term temporal fusion for multi-view 3D perception," *IEEE Robot. Automat. Lett.*, vol. 9, no. 7, pp. 6544–6551, Jul. 2024.

[58] Y. Zhou et al., "End-to-end multi-view fusion for 3D object detection in LiDAR point clouds," in *Proc. Conf. Robot Learn.*, 2020, pp. 923–932.

[59] Y. Zhang et al., "Optimal vehicle path planning using quadratic optimization for baidu apollo open platform," in *Proc. IEEE Intell. Veh. Symp.*, 2020, pp. 978–984.

[60] H. Zhang et al., "DINO: DETR with improved denoising anchor boxes for end-to-end object detection," in *Proc. Int. Conf. Learn. Representations*, 2023, pp. 1–18.

[61] F. Li et al., "DN-DETR: Accelerate DETR training by introducing query denoising," in *IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 13619–13627.

[62] D. Meng et al., "Conditional DETR for fast training convergence," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2021, pp. 3651–3660.

[63] T.-Y. Lin et al., "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis*, 2017, pp. 2980–2988.

[64] X. Weng et al., "3D multi-object tracking: A baseline and new evaluation metrics," in *Proc. IEEE Int. Conf. Intell. Robots Syst.*, 2020, pp. 10359–10366.

[65] B. Zhu et al., "Class-balanced grouping and sampling for point cloud 3D object detection," 2019, *arXiv:1908.09492*.

[66] Z. Tang et al., "Strong detector with simple tracker," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 3047–3053.

[67] L. Wang et al., "CAMO-MOT: Combined appearance-motion optimization for 3D multi-object tracking with camera-LiDAR fusion," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 11, pp. 11981–11996, Nov. 2023.

[68] C. Yang et al., "Bevformer v2: Adapting modern image backbones to bird's-eye-view recognition via perspective supervision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 17830–17839.

[69] F. Liu et al., "Ray denoising: Depth-aware hard negative sampling for multi-view 3D object detection," in *Proc. Eur. Conf. Comput. Vis.*, 2024, pp. 200–217.

[70] J. Hou et al., "Open: Object-wise position embedding for multi-view 3D object detection," in *Proc. Eur. Conf. Comput. Vis.*, 2024, pp. 146–162.

[71] J. Zhang et al., "GeoBEV: Learning geometric BEV representation for multi-view 3D object detection," in *Proc. AAAI Conf. Artif. Intell.*, 2025, pp. 9960–9968.

[72] Z. Li et al., "BEVNeXt: Reviving dense bev frameworks for 3D object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 20113–20123.

[73] D. Kwon et al., "Memdistill: Distilling LiDAR knowledge into memory for camera-only 3D object detection," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2025, pp. 6828–6838.

[74] D. Park et al., "Is pseudo-LiDAR needed for monocular 3D object detection?," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2021, pp. 3142–3152.

[75] Z. Zong et al., "Temporal enhanced training of multi-view 3D object detector via historical object prediction," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 3781–3790.

[76] Y. Lee et al., "An energy and GPU-computation efficient backbone network for real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops*, 2019, pp. 1–9.

[77] Z. Liu et al., "A convnet for the 2020s," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 11976–11986.

[78] W. Wang et al., "Internimage: Exploring large-scale vision foundation models with deformable convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 14408–14419.

[79] M. Ji et al., "OcRFDet: Object-centric radiance fields for multi-view 3D object detection in autonomous driving," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2025, pp. 24933–24942.

[80] Y. Fang et al., "EVA-02: A visual representation for neon genesis," *Image Vis. Comput.*, vol. 149, 2024, Art. no. 105171.

[81] Z. Chen et al., "Deformable feature aggregation for dynamic multi-modal 3D object detection," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 628–644.

[82] K. He et al., "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.

[83] J. Yang et al., "Qtrack: Embracing quality clues for robust 3D multi-object tracking," in *Proc. IEEE Int. Conf. Intell. Robots Syst.*, 2024, pp. 4904–4911.

[84] Q. Lian et al., "DORT: Modeling dynamic objects in recurrent for multi-camera 3D object detection and tracking," in *Proc. Conf. Robot Learn.*, 2023, pp. 1–17.

[85] Y. Li et al., "End-to-end 3D tracking with decoupled queries," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 18302–18311.

[86] T. Tang et al., "S2-track: A simple yet strong approach for end-to-end 3D multi-object tracking," in *Proc. Int. Conf. Mach. Learn.*, 2025, pp. 1–17.

[87] Z. Pang et al., "Standing between past and future: Spatio-temporal modeling for multi-camera 3D multi-object tracking," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 17928–17938.

[88] S. Ding et al., "ADA-track: End-to-end multi-camera 3D multi-object tracking with alternating detection and association," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 15184–15194.

[89] T. Wang et al., "FCOS3D: Fully convolutional one-stage monocular 3D object detection," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2021, pp. 913–922.
[90] K. Guo and Q. Ling, "PromptDet: A lightweight 3D object detection framework with LiDAR prompts," in *Proc. AAAI Conf. Artif. Intell.*, 2025, pp. 3266–3274.
[91] T. Yin et al., "Center-based 3D object detection and tracking," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 11784–11793.
[92] Y. Chen et al., "VoxelNext: Fully sparse voxelnet for 3D object detection and tracking," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 21674–21683.
[93] L. Fan et al., "FSD v2: Improving fully sparse 3D object detection with virtual voxels," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 2, pp. 1279–1292, Feb. 2025.
[94] X. Sun et al., "Spatial awareness enhancement based single-stage anchor-free 3D object detection for autonomous driving," *Displays*, vol. 85, 2024, Art. no. 102821.
[95] G. K. Erabati and H. Araujo, "DAFDeTr: Deformable attention fusion based 3D detection transformer," in *Proc. Int. Conf. Robot. Comput. Vis. Intell. Syst.*, 2024, pp. 293–315.
[96] J. Ren et al., "High performance point-voxel feature set abstraction with mamba for 3D object detection," *Expert Syst. Appl.*, vol. 286, 2025, Art. no. 128127.
[97] Z. Ding et al., "AS-Det: Active sampling for adaptive 3D object detection in point clouds," in *Proc. AAAI Conf. Artif. Intell.*, 2025, pp. 2762–2770.
[98] Y. Li et al., "Unifying voxel-based representation with transformer for 3D object detection," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 18442–18455.
[99] Z. Yang et al., "Deepinteraction: 3D object detection via modality interaction," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 1992–2005.
[100] Z. Song et al., "Robofusion: Towards robust multi-modal 3D object detection via sam," in *Proc. Int. Joint Conf. Artif. Intell.*, 2024, pp. 1272–1280.
[101] K. Li et al., "PF3Det: A prompted foundation feature assisted visual LiDAR 3D detector," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshop*, 2025, pp. 3778–3787.
[102] P. Sun et al., "Scalability in perception for autonomous driving: Waymo open dataset," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 2446–2454.

Xuewu Lin received the master's degree from the School of Vehicle and Mobility, Tsinghua University, Beijing, China, in 2021. He is currently working as a researcher with the Robot Lab of Horizon Robotics. His current research interests include embodied intelligence and autonomous driving.

Zixiang Pei received the master degree from the State Key Lab of Networking and Switching Technology, Beijing University of Posts and Telecommunications, in 2022, advised by Prof. Rongheng Lin. He is currently employed as a computer vision algorithm engineer with Horizon Robotics. His research interests include 3D perception, online map construction, and multi-task learning in autonomous driving.

Keyu Li received the BE degree in electronic information engineering from the School of Telecommunications Engineering, Xidian University, Xi'an, China, in 2023, advised by Prof. Nannan Wang. He is currently a research engineer with the Robotics Laboratory, Horizon Robotics. His research interests include 3D perception, autonomous driving systems, and embodied intelligence.

Tianwei Lin received the master's degree from Shanghai Jiao Tong University, in 2019, advised by Prof. Xu Zhao. He is currently works with Horizon Robotics as a computer vision research scientist. His research interests include: autonomous driving, embodied AI, cross-modal understanding, and generation.

Lichao Huang received the master degree from the Department of Computing, Imperial College, London, in 2014. He is currently a researcher with Horizon Robotics. His research interests include deep learning for computer vision and autonomous robots.

Chenyao Yu is currently working toward the graduate degree majoring in artificial intelligence with Soochow University since 2024. Before that, she interned with the Eco-Development Department and Technical Solution Delivery Department, Horizon Robotics. She is mainly engaged in computer vision and intelligent driving research.

John See received the BEng, MEngSc, and PhD degrees from Multimedia University, Malaysia. He is an associate professor with the School of Mathematical and Computing Sciences, Heriot-Watt University Malaysia. He is also an award recipient of the Belt and Road Initiative Young Scientist Exchange Fellowship where he was a visiting research fellow to Shanghai Jiao Tong University, China.

Zhizhong Su received the BE degree from Shanghai Jiao Tong University, China, and the MS degree from Indiana University Bloomington, USA. He is currently the director of Robot Lab, Horizon Robotics. His research interests include autonomous driving and robotics learning.

Cong Yang received the PhD degree in computer vision and pattern recognition from the University of Siegen (Germany), in 2016. He is an associate professor with Soochow University since 2022. Before that, he was a postdoc researcher with the MAGRIT team in INRIA (France). Later, he worked scientifically and led the computer vision and machine learning teams in Clobotics and Horizon Robotics. His main research interests include computer vision, pattern recognition, and their interdisciplinary applications.